MANUFACTURING & SERVICE OPERATIONS MANAGEMENT
https://pubsonline.informs.org/journal/msom

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *Manufacturing & Service Operations Management*

ISSN 1523-4614, EISSN 1526-5498

Data-Driven Threshold Scheduling for LLM Inference: Preventing Memory Eviction Cascades

(Authors' names are not included for peer review)

Abstract. Large Language Models (LLMs) power modern interactive services including chatbots, virtual assistants, and programming support tools, serving millions of customer queries daily. These AI-powered services face unique operational challenges: managing stochastic customer demand, ensuring quality of service (QoS) under strict resource constraints, and balancing service capacity with responsiveness. The Key-Value (KV) cache grows dynamically during service provision, and memory overflow triggers eviction cascades that can cause system-wide failures. Even when resource capacity exceeds theoretical requirements, conventional scheduling fails because it does not account for this dynamic resource consumption—a system that should be stable can become unstable under poor scheduling.

We formulate LLM inference as a multi-stage service scheduling problem under memory constraints, where customer requests (prompts) arrive stochastically and service resource requirements (KV cache) grow dynamically during processing. Using fluid dynamics from queueing theory, we develop the Waiting for Accumulated Inference Threshold (WAIT) algorithm that prevents eviction cascades by keeping the system near load balance, achieving near-optimal service capacity while maintaining bounded customer waiting time and response latency.

For practical service settings where output lengths are unknown at arrival, we introduce Nested WAIT. Rather than predicting service times, Nested WAIT classifies customer requests on-the-fly: short requests complete early and exit, while longer requests naturally advance to later service stages. A safety buffer provides high-probability protection against resource overflow with only logarithmic overhead.

Theoretical analysis establishes near-optimal performance in the asymptotic regime. Experiments with real-world service data (LMSYS-Chat-1M) on NVIDIA A100 GPUs demonstrate 12-25% higher service throughput and 30% lower customer latency compared to current serving systems (vLLM, Sarathi). This work bridges service operations theory with AI systems, offering scheduling strategies directly applicable to emerging LLM-powered services.

Key words: Large Language Model, Service Operations, Key-value Cache, Memory Constraint, Online Scheduling

1. Introduction

Large Language Models (LLMs) power chatbots (Anthropic 2023, OpenAI 2024), search engines (Google 2023), and coding assistants (GitHub 2022). Inference—generating responses to queries—imposes significant memory and compute demands (García-Martín et al. 2019). Efficient scheduling is critical: ChatGPT incurs daily costs exceeding $700,000 (Patel 2023).

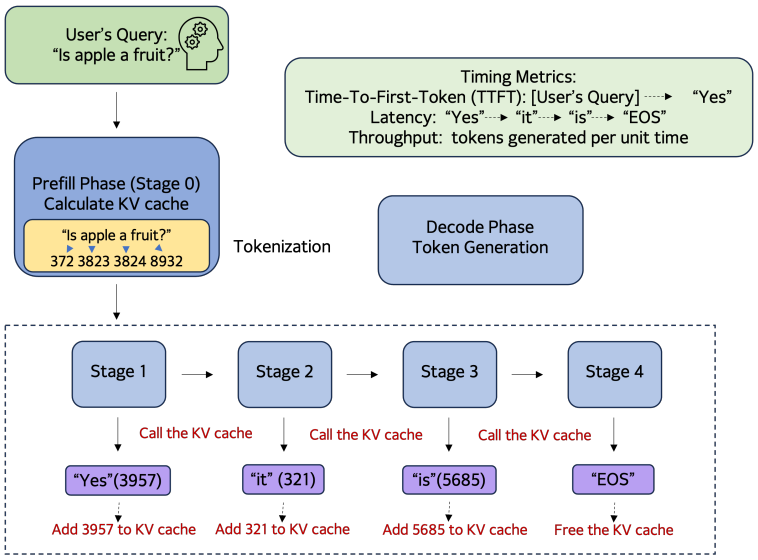

Figure 1 LLM inference service process.

LLM inference (Figure 1) proceeds in two phases. In the **prefill** phase, the model tokenizes input and builds a Key-Value (KV) cache storing attention keys and values. In the **decode** phase, output tokens are generated sequentially, with each step querying and updating the KV cache. KV cache memory grows linearly with output length, creating resource demands that change during processing.

The KV cache avoids quadratic recomputation but creates capacity challenges: memory grows unpredictably, and exceeding GPU limits degrades performance (Kang et al. 2024). Batching optimizes GPU use, but traditional scheduling (e.g., Shortest Job First) assumes fixed job sizes and known processing times—inapplicable when KV cache grows dynamically and output lengths are unknown. Classical OR approaches fail for this multi-phase, memory-constrained setting.

Recent system-level optimizations (Yu et al. 2022, Kwon et al. 2023, Agrawal et al. 2023, Pope et al. 2023, DeepSeek-AI 2024, Patel et al. 2024, Zhong et al. 2024) focus on engineering solutions but lack rigorous mathematical foundations. Output length predictions are imprecise or costly (Fu et al. 2025). When output lengths are unknown, algorithm performance degrades (Proposition 5).

We develop a fluid dynamics approximation to establish a tractable benchmark. From this benchmark, we derive the WAIT algorithm for known output lengths and Nested WAIT for unknown lengths. Nested WAIT uses hierarchical thresholds to manage random arrivals without predicting service times; prompts classify themselves on-the-fly by completing or continuing at segment boundaries. Both algorithms achieve asymptotic near-optimality.

1.1. Summary of Contributions

We make three contributions. First, we formulate LLM inference as a multi-stage online scheduling problem under memory constraints (Section 2) and employ fluid dynamics to establish a tractable performance benchmark (Section 3). Second, we develop WAIT for known output lengths (Section 4) and Nested WAIT for unknown lengths (Section 5), proving asymptotic optimality via coupling analyses; extensions appear in Section 7. Third, experiments on LMSYS-Chat-1M with Llama2-7B demonstrate 12–25% throughput improvement and 30% latency reduction over vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023) (Section 6).

1.2. Other Related Work

Online Scheduling and Queueing. Classical online scheduling Devanur and Hayes (2009), Vee et al. (2010), Cole and Roughgarden (2014), Balkanski et al. (2016), Lattanzi et al. (2020), Im and Moseley (2013), Lucier et al. (2013), Liu and Lu (2015), Li et al. (2020) and queueing theory use fluid models for near-optimal control Mandelbaum et al. (1998), Maglaras (2000), Bäuerle (2002), Liu and Whitt (2011). However, LLM inference differs fundamentally: resource requirements grow dynamically (KV cache) and involve multi-phase processing (prefill/decode), unlike static single-phase jobs in classical scheduling.

LLM Inference. Most LLM inference work focuses on system optimizations Patel (2023), Zhong et al. (2024), Yu et al. (2022), Agrawal et al. (2023, 2024b), DeepSeek-AI (2024), Fu et al. (2025) rather than theoretical analysis. Unknown output lengths cause significant performance degradation (Proposition 5). We develop a scheduling framework using fluid dynamics and coupling techniques to achieve asymptotic optimality under unknown lengths. Li et al. (2025) propose a similar linear batch processing model achieving optimal throughput for work-conserving policies, but do not address KV cache memory management or latency metrics (TTFT, end-to-end latency) critical for interactive applications.

1.3. Notations

For integer $n \geq 1$, we denote $[n] = \{1, 2, \ldots, n\}$ as the set of integers from 1 to n. For $x \in \mathbb{R}$, denote $\lceil x \rceil$ as the smallest integer not smaller than x and $\lfloor x \rfloor$ as the largest integer not greater than x. Denote $x_+ = \max\{x, 0\}$. For set S, denote $|S|$ as its cardinality. For two functions $f(T)$ and $g(T)$, we use $f(T) = O(g(T))$ if there exists constant $c_1 > 0$ such that $f(T) \leq c_1 g(T)$ as $T \to +\infty$ and $f(T) = \Omega(g(T))$ if there exists constant $c_2 > 0$ such that $f(T) \geq c_2 g(T)$ as $T \to +\infty$.

2. Model

We formalize the online scheduling problem for Large Language Model (LLM) inference under memory constraints. Unlike traditional scheduling where job sizes are fixed, LLM inference involves *dynamically growing memory demands*: the Key-Value (KV) cache expands continuously as prompts generate output tokens, and the two-phase structure (prefill and decode) creates fundamentally different resource consumption patterns. This dynamic growth, combined with unknown output lengths, distinguishes our problem from classical queueing models. We first introduce the inference process, then describe prompt characteristics, batching rules, system constraints, and performance metrics.

2.1. Notation

We summarize the key notation used throughout this paper in Table 1.

Table 1 Summary of Notation

Symbol	Description
m	Number of prompt types
j	Index of prompt type, $j \in \{1, \ldots, m\}$
λ_j	Arrival rate of type-j prompts (Poisson process)
l_j	Input length (tokens) of type-j prompts after prefill
l'_j	Output length (tokens) of type-j prompts in decode phase
s	Stage index: $s = 0$ for prefill, $s \in \{1, \ldots, l'_j\}$ for decode
k	Segment index in Nested WAIT, $k \in \{1, \ldots, m\}$
b	Batch index (iteration count)
n^t_{js}	Number of type-j prompts at stage s at time t
n_j	Threshold for type-j prompts in WAIT algorithm
n_k	Threshold for segment k in Nested WAIT
C	GPU memory capacity
$\mathcal{G}^t$	Set of prompts with KV cache on GPU at time t
B^t	Batch of prompts processed at iteration t ($B^t \subseteq \mathcal{G}^t$)
τ	Iteration time to process a batch
d_0	Fixed overhead time per batch iteration
d_1	Time cost per unit of KV cache memory
M^*	Equilibrium memory usage in fluid model
n^*_j	Equilibrium number of active type-j prompts
Throughput*	Equilibrium throughput (fluid benchmark)
Π	Class of admissible scheduling policies

2.2. Prompts and Inference Process

User queries, called *prompts*, arrive at a single Graphics Processing Unit (GPU) for processing. We focus on maximizing how fully the GPU is used, and our algorithms extend to multi-GPU settings such as pipeline parallelism, where each prompt is processed across different GPUs in different

layers of the LLM. When communication costs are negligible, our algorithms apply directly to these settings.

During inference, each prompt is divided into discrete units called *tokens* during the prefill phase, then generates output text in the decode phase. A token represents a distinct text element, such as a word or subword, and its processing relies on a memory structure called the Key-Value (KV) cache. The KV cache improves inference efficiency by storing the computational history of prior tokens, thus avoiding redundant computations. The cache size increases as tokens are processed, requiring careful memory management to maintain system performance.

Prompt Characteristics: Prompts arrive stochastically and are classified into m distinct types according to their input / output lengths, indexed by $j \in \{1, \ldots, m\}$. We assume that prompts of type j arrive according to a Poisson process with rate λ_j. Each prompt's memory footprint depends on its processing stage. A prompt of type j has the following properties:

$$\begin{cases} \text{a length of } l_j \text{ tokens after prefill;} \\ \text{a length of } l_j + l_j' \text{ tokens after decode.} \end{cases}$$

Therefore, a type-j prompt must undergo one prefill iteration followed by l_j' decode iterations to complete its inference processing. During the prefill phase, the prompt's input context and question are embedded as l_j tokens in a single iteration, consuming a KV cache of size l_j units. In the subsequent decode phase, one output token is generated per iteration, each incurring an additional unit of KV cache memory.

To track processing progress, we define a prompt to be in **stage** $s = 0$ if it is about to undergo the prefill phase, and in **stage** s (where $s \in \{1, \ldots, l_j'\}$) if it is scheduled to receive its s-th output token during the decode phase. Thus, a prompt at stage s occupies memory of size $l_j + s$ units.

Though the total number of different types m can be large (for example, from 1 to 10000), our algorithms and their theoretical guarantees exhibit only a weak (logarithmic) dependence on m. The analysis relies primarily on the total arrival rate across all types, ensuring that our approach scales efficiently regardless of the number of types.

While our analysis focuses on fixed arrival rates λ_j, our threshold-based algorithms can be extended to settings with time-varying arrivals by dynamically adjusting thresholds based on the arrival rates (see Section 7).

2.3. Batching and Iteration Time

To use GPU resources efficiently, the GPU processes tasks in *batches*, combining prompts for prefilling and prefilled prompts for decoding. Figure 2 illustrates this: prompts $P1$ and $P2$ arrive sequentially and are batched according to their current stage (prefill or decode). The GPU first processes $P1$'s prefill, then $P2$'s prefill. Subsequently, both enter decode, generating one token per iteration in parallel batches. This interleaving of prefill and decode stages improves throughput while managing memory efficiently.

In general, a batch contains two groups: n_1 prompts in the prefill phase (stage $s = 0$), where prompt i has input length $l_{j(i)}$ tokens corresponding to its type $j(i)$ and is processed to initialize its KV cache; and n_2 prompts in the decode phase, where prompt i is at stage $s_i \in \{1, \ldots, l'_{j(i)}\}$, having generated s_i output tokens, with total KV cache size $l_{j(i)} + s_i$ tokens.

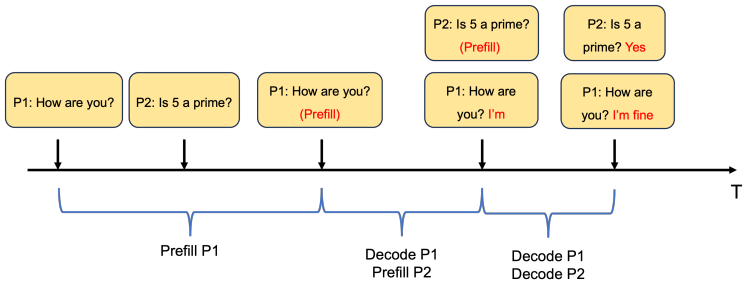

Figure 2 Example of batching and scheduling.

Experiments in Agrawal et al. (2023), Kwon et al. (2023), Zhong et al. (2024) show that *iteration time* scales linearly with total KV cache size. We model this as:

$$\tau = d_0 + d_1 \cdot \underbrace{\left(\sum_{i=1}^{n_1} l_{j(i)} + \sum_{i'=1}^{n_2} (l_{j(i')} + s_{i'}) \right)}_{\text{Total KV cache size}}, \tag{1}$$

where d_0 denotes the fixed overhead time associated with processing a batch, and d_1 represents the time cost per unit of KV cache memory. Equation (1) shows that iteration time scales linearly with total KV cache size. The fixed overhead d_0 has an important implication: *to maximize throughput, we should use memory as fully as possible*. Larger batches amortize the fixed cost d_0 across more prompts, improving efficiency. However, this must be balanced against the memory constraint (2) to avoid eviction. Figure 3 provides a numerical validation of this modeling. We deploy Llama-7B

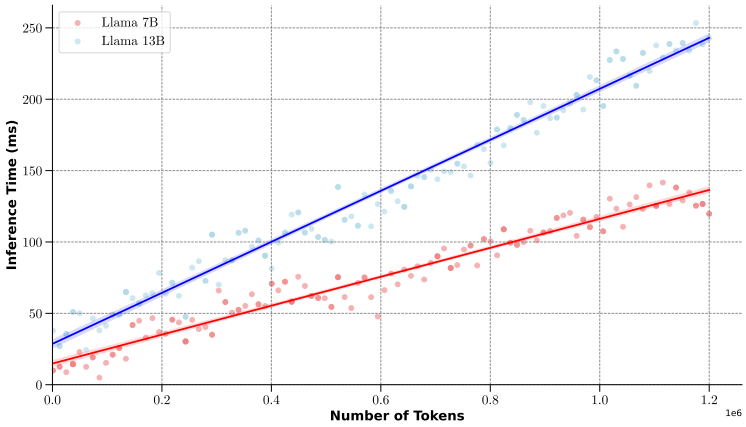

Figure 3 **Inference time of Llama-7B and Llama-13B with different number of tokens in the batch on a single L20 GPU**

and Llama-13B on a single L20 GPU to test time cost per iteration, using batches consisting of different number of tokens repeated 20 times. The result is highly consistent with our model.

The linear model (1) is most accurate when: (i) batches are decode-dominated, as each decode iteration processes one token per prompt and the computational cost is dominated by KV cache memory access; (ii) the system employs prefill-decode disaggregation (Zhong et al. 2024, Patel et al. 2024), where the decode server handles only decode operations, making Eq. (1) exact. In mixed prefill-decode batches, the linear model provides a good approximation when prefill lengths are similar across prompts.

More generally, iteration time can exhibit piecewise linear behavior (e.g., Li et al. (2025)), capturing transitions between compute-bound and memory-bound regimes. Our threshold-based algorithms extend naturally to such models, as the key insight—balancing arrivals and completions—remains valid when iteration time scales predictably with batch composition. We adopt the simplified linear model (1) for tractability, validated by Figure 3 in the decode-dominated regime.

2.4. Memory Constraint and Challenges

Unlike traditional scheduling where job sizes are fixed, LLM inference faces a unique challenge: prompts occupy memory that **grows during decode**. Each generated token adds one unit to the prompt's KV cache, causing the memory footprint to expand continuously until completion. This dynamic growth creates eviction risks even when total capacity appears sufficient.

Memory Constraint: At all times t, the total KV cache stored on GPU must not exceed capacity C:

$$\sum_{i \in \mathcal{G}^t} (l_{j(i)} + s_i^t) \le C, \tag{2}$$

where $\mathcal{G}^t$ denotes the set of prompts whose KV caches are stored on GPU at time t. This includes both prompts actively processed in the current batch $B^t \subseteq \mathcal{G}^t$ and prompts that are preempted but retain their KV caches on GPU. Here $j(i)$ is the type of prompt i, and $s_i^t \in \{0, 1, \ldots, l'_{j(i)}\}$ is its current stage. Unlike traditional scheduling where job sizes are fixed, the memory footprint $l_{j(i)} + s_i^t$ *grows dynamically* as decode progresses (s_i^t increases each iteration). This dynamic growth creates the risk of *eviction*: if memory becomes insufficient to satisfy the constraint (2), ongoing prompts must be removed from the GPU and their KV caches discarded. When eviction is necessary, prompts are removed in **LIFO order** (most recently admitted prompts are evicted first) until sufficient memory is freed. Evicted prompts lose all accumulated computation and must restart from the prefill phase (stage $s = 0$), re-entering the scheduling queue as new arrivals. This creates a vicious cycle: eviction temporarily frees memory, but restarted prompts compete for resources again, potentially triggering further evictions—a phenomenon we call *cascading failure*. The following example illustrates this mechanism.

EXAMPLE 1 (DYNAMIC MEMORY GROWTH AND EVICTION). Consider prompts with $(l, l') = (1, 1)$: after prefill, memory is 1 unit; after decode, 2 units. With capacity $C = 12$, a stable batch processes 4 prefills (4 units) and 4 decodes (8 units), totaling 12 units.

Now suppose 6 prompts arrive when 3 are decoding (occupying 6 units). Admitting all 6 for prefill fills memory to $6 \times 2 = 12$ units after the 3 existing prompts complete. When 4 new prompts arrive next, prefilling them requires evicting 2 decode prompts to free 4 units. These evicted prompts restart from stage $s = 0$, re-entering the queue. The restarted prompts compete with new arrivals, perpetuating the imbalance and triggering repeated evictions—a **cascading failure**.

This example reveals the core challenge our algorithms target. Even with $C \ge M^*$ (equilibrium memory from Section 3), poor admission control makes the system unstable. The eviction-restart cycle can persist indefinitely, degrading throughput by 12–25% (Example 2 quantifies this for FCFS).

Core Challenge. When output lengths are unknown, optimistic admission risks overflow; pessimistic admission wastes capacity. Our threshold-based algorithms resolve this dilemma by: (i) preventing evictions through controlled admission, (ii) exploiting memory fully via batching, (iii) classifying prompts on-the-fly as they complete decode stages. This approach maintains the system near the load-balanced equilibrium derived in Section 3, where arrivals match completions and memory use is stable.

2.5. Performance Metrics

We optimize **throughput** (average tokens generated per unit time, counted after prompt completion), subject to constraints on **latency** (average time from arrival to completion) and **TTFT** (time to first token, measuring initial user-perceived delay).

2.6. Optimization Problem and Policy Space

We formalize the problem as optimizing throughput subject to latency and TTFT constraints over continuous time horizon $[0, T]$:

Policy Space Π. We define the class of admissible scheduling policies Π. At each decision epoch t, a policy $\pi \in \Pi$ observes the system state and makes scheduling decisions subject to the following constraints:

Information Structure. The system state at time t includes: (i) the set of all prompts that have arrived by time t, along with their input lengths l_i; (ii) the current stage s_i^t of each prompt i (i.e., how many output tokens have been generated); (iii) whether each prompt's KV cache is currently stored on GPU. Policies are *non-anticipating*: decisions at time t depend only on information available at or before time t. In particular, the output length l_i' of a prompt may be unknown until the prompt completes (generates an end-of-sequence token). This unknown output length is a key challenge addressed by our Nested WAIT algorithm (Section 5).

Admissible Actions. At each decision epoch, a policy may:

1. **Admit**: Select a subset of waiting prompts (at stage $s = 0$) to enter the system. Admitted prompts wait until included in a batch for prefill.
2. **Batch**: Form a batch B^t of prompts to process in the current iteration. After prefill completes, a prompt's KV cache is allocated on GPU.
3. **Preempt**: Pause a prompt while retaining its KV cache on GPU. The prompt can resume later without recomputation.

4. **Evict**: Remove a prompt's KV cache from GPU to free memory when the constraint (2) cannot be satisfied. Eviction follows a **LIFO (Last In, First Out)** policy: the most recently admitted prompts are evicted first. The evicted prompt returns to the waiting queue at stage $s = 0$, losing all prior computation.

Memory Constraint. At all times, the total KV cache stored on GPU must satisfy the capacity constraint (2).

Preemption and eviction are both permitted, but eviction is costly: the evicted prompt re-enters the system as a new arrival and must repeat all processing stages from the beginning. Our WAIT algorithms are designed to *prevent eviction* by controlling admission through thresholds, thereby avoiding the memory pressure that causes eviction.

3. Fluid Dynamics and Equilibrium

Unlike traditional queueing systems, LLM inference features dynamically growing memory: KV cache expands as prompts progress through decode stages. This makes predicting overflow difficult; when total memory exceeds capacity C, evictions waste partial computation, degrading throughput and latency.

We develop a fluid model characterizing equilibrium—a balanced state where arrivals match completions. This equilibrium serves two purposes: it quantifies the maximum sustainable throughput **Throughput*** under memory constraint C, and it reveals the load-balancing principle guiding our algorithm (Section 4). The principle is simple: maintain the system near equilibrium to prevent evictions. We first analyze single-type systems, then extend to multiple heterogeneous types.

3.1. Single-Type Fluid Model

For a single type j, each prompt uses l_j tokens of memory at prefill stage ($s = 0$) and $l_j + s$ tokens at decode stage $s = 1, \ldots, l'_j$. At equilibrium, the system maintains n^*_j active prompts evenly distributed across $(l'_j + 1)$ stages. This balanced distribution minimizes overflow risk: if prompts accumulate at later stages (higher memory), total usage may exceed C, forcing evictions.

The system becomes unstable when arrivals exceed the inverse of total processing time $d_1(l'_j + 1)(l_j + l'_j/2)$.

PROPOSITION 1. *When the condition* $\lambda_j d_1 (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) \geq 1$ *is satisfied, the system becomes unstable in the sense that the expected average latency under any scheduling policy with deterministic arrival rate* λ_j *grows linearly with time. Formally,*

$$\mathbb{E}[\textbf{\textit{Latency}}^{(T,\pi)}] = \Omega(T) \quad as \quad T \to \infty.$$

Proof in Appendix EC.1.1. We assume $\lambda_j d_1 (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) < 1$ henceforth, ensuring equilibrium exists.

Total memory at equilibrium is:

$$M_j^* = \frac{n_j^*}{l'_j + 1} \sum_{s=0}^{l'_j} (l_j + s) = n_j^* \left(l_j + \frac{l'_j}{2}\right), \tag{3}$$

where $(l_j + l'_j/2)$ is average memory per prompt. At equilibrium, arrivals equal completions:

$$\underbrace{(d_0 + d_1 M_j^*)}_{\text{Time per iteration}} \lambda_j = \left(d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2}\right)\right) \lambda_j = \frac{n_j^*}{l'_j + 1}.$$

Solving for n_j^* yields:

$$n_j^* = \frac{d_0 \lambda_j (l'_j + 1)}{1 - d_1 \lambda_j (l'_j + 1)(l_j + \frac{l'_j}{2})}.$$

Equilibrium memory is:

$$M_j^* = n_j^* \left(l_j + \tfrac{l'_j}{2}\right) = \frac{d_0 \lambda_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)}{1 - d_1 \lambda_j (l'_j + 1)(l_j + \frac{l'_j}{2})}. \tag{4}$$

Equilibrium throughput is:

$$\begin{aligned} \textbf{Throughput}_j^* &= \frac{n_j^* \cdot l'_j}{(l'_j + 1)(d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2}\right))} \\ &= \lambda_j l'_j, \end{aligned} \tag{5}$$

where the equality follows from the equilibrium condition.

3.2. Multiple-Type Fluid Model

Real systems handle multiple prompt types with different lengths l_j, l'_j, competing for memory C. Admitting long prompts may force evicting multiple short prompts—a cascading failure analyzed in Section 4.

With n_j^* prompts of type j in equilibrium, total memory is:

$$M^* = \sum_{j=1}^{m} n_j^* \left(l_j + \frac{l'_j}{2} \right) \tag{6}$$

where $(d_0 + d_1 M^*)\lambda_j (l'_j + 1) = n_j^*$ for all j. This yields:

$$M^* = \frac{d_0 \sum_{j=1}^{m} \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \sum_{j=1}^{m} \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)} \tag{7}$$

The processing time per iteration in equilibrium is:

$$\begin{aligned} &\text{Processing Time}^* \\ &= d_0 + d_1 M^* \\ &= \frac{d_0}{1 - d_1 \sum_{j=1}^{m} \lambda_j (l'_j + 1)(l_j + \frac{1}{2} l'_j)}, \end{aligned} \tag{8}$$

as well as the average throughput:

$$\begin{aligned} \textbf{Throughput}^* &= \sum_{j=1}^{m} \frac{n_j^* \cdot l'_j}{(l'_j + 1) \cdot (d_0 + d_1 M^*)} \\ &= \sum_{j=1}^{m} \lambda_j l'_j \end{aligned} \tag{9}$$

The equilibrium throughput $\textbf{Throughput}^*$ benchmarks all non-predictive policies in Π.

PROPOSITION 2. *Suppose that the memory capacity satisfies $C \geq M^*$, where M^* denotes the equilibrium memory given in (7). Then, for any online policy $\pi \in \Pi$, the expected throughput satisfies*

$$\mathbb{E}[\textbf{\textit{Throughput}}^{(T,\pi)}] \leq \textbf{\textit{Throughput}}^*,$$

where $\textbf{\textit{Throughput}}^*$ is given in (9).

This quantifies the theoretical limit and reveals a design principle: policies maintaining balanced prompt distribution across stages can approach $\textbf{Throughput}^*$. Our WAIT algorithm (Section 4) uses thresholds to keep the system near equilibrium, preventing cascading failures that plague FCFS (Example 2).

4. Scheduling with Known Arrival Types: The WAIT Algorithm

This section introduces the *Waiting for Accumulated Inference Threshold* (WAIT) algorithm, a scheduling policy that maximizes throughput when prompt arrival types are known. Building on the fluid dynamics from Section 3, WAIT optimizes batch formation by accumulating prompts until specific thresholds are met. This approach maintains the system near equilibrium, respecting memory constraints while optimizing latency and time-to-first-token (TTFT). We assume that the memory capacity $C \geq M^*$, the equilibrium memory capacity given in (7) (otherwise the throughput can never approach the optimal fluid throughput (9) by Proposition 2).

REMARK 1 (ON THE ASSUMPTION $C \geq M^*$). Our theoretical analysis assumes $C \geq M^*$, as this is the necessary condition under which the system can achieve stable operation without accumulating backlog. When $C < M^*$ (the overloaded regime), our algorithms remain effective: the eviction prevention mechanism still outperforms baselines, as demonstrated in Section 6. Avoiding eviction cascades provides value across all operating regimes.

4.1. Motivation: Why Naive Scheduling Fails

We first demonstrate why naive scheduling policies fail even when memory capacity is sufficient, motivating the design of WAIT.

Consider what happens under a First-Come-First-Serve (FCFS) policy that processes prompts immediately upon arrival. Even when the memory capacity C equals the equilibrium memory M^*—seemingly sufficient to handle the workload—FCFS can trigger catastrophic failure. The following proposition formalizes this observation.

PROPOSITION 3. *When the memory capacity C equals the equilibrium memory M^*, there exists an instance where the First-Come-First-Serve (FCFS) policy incurs a throughput gap of* $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^T] = \Omega(1)$ *as* $T \to \infty$.

The proof is provided in Appendix. To illustrate the mechanism behind this failure concretely, consider the following example.

EXAMPLE 2 (EVICTION CASCADE UNDER FCFS). Consider a single prompt type where each prompt is a simple query such as "Hello" with a 1-token response "Hi"—prefill length $l = 1$ and decode length $l' = 1$. Each prompt occupies 1 unit of memory after prefill and 2 units after decode. Let the memory capacity be $C = 12$ and arrival rate $\lambda = 4$. The fluid equilibrium has 4

prompts at each stage (prefill and decode), using exactly $4 \times 1 + 4 \times 2 = 12$ units of memory with 4 completions per iteration.

Now suppose the system starts slightly imbalanced with 6 prompts at stage 0 (awaiting prefill) and 3 prompts at stage 1 (in decode). Under FCFS:

1. **Iteration 1**: Process all 9 prompts. The 3 decode prompts complete, but the 6 prefill prompts advance to decode, requiring $6 \times 2 = 12$ units. Memory is exactly full with only 3 completions.
2. **Iteration 2**: During iteration 1, approximately 4 new prompts arrive. To admit them for prefill ($4 \times 1 = 4$ units needed), we must evict 2 decode prompts (freeing $2 \times 2 = 4$ units). Now we have 4 prefill + 4 decode = 4 + 8 = 12 units.
3. **Iteration 3**: The 4 decode prompts complete (4 completions). The 2 previously evicted prompts, having lost their KV caches, must now restart from the prefill phase (stage $s = 0$)—they re-enter the queue as new arrivals, competing for resources alongside genuinely new prompts. This restart mechanism perpetuates the imbalance.

This eviction-restart cycle persists, yielding approximately 3-3.5 completions per iteration instead of the optimal 4. **Even though $C = M^* = 12$ (theoretically sufficient capacity), FCFS causes cascading failures that reduce throughput by 12-25%.**

This example demonstrates a fundamental insight: *memory sufficiency alone does not guarantee stability*—a system that should be stable can become unstable under poor scheduling. The root cause is that FCFS does not maintain *load balance*: it admits prompts greedily, causing the system to deviate from the fluid equilibrium and triggering the eviction-restart cycle.

4.2. The WAIT Algorithm

The WAIT algorithm prevents eviction cascades by maintaining load balance through a threshold mechanism. The core idea is to control admission at each decode stage, ensuring that the rate of new arrivals matches the rate of completions. This keeps the system close to the fluid equilibrium derived in Section 3, *approaching load balance* where arrival and completion rates match. This principle—maintaining the system near equilibrium through controlled admission—is central to preventing the cascading failures illustrated in Example 2.

The threshold-and-wait mechanism achieves three goals: (1) prevents *eviction cascades* by controlling admission; (2) *decouples multi-type dynamics* into m independent analyses, breaking inter-type dependencies through algorithm design; (3) *exploits memory fully* by forming batches

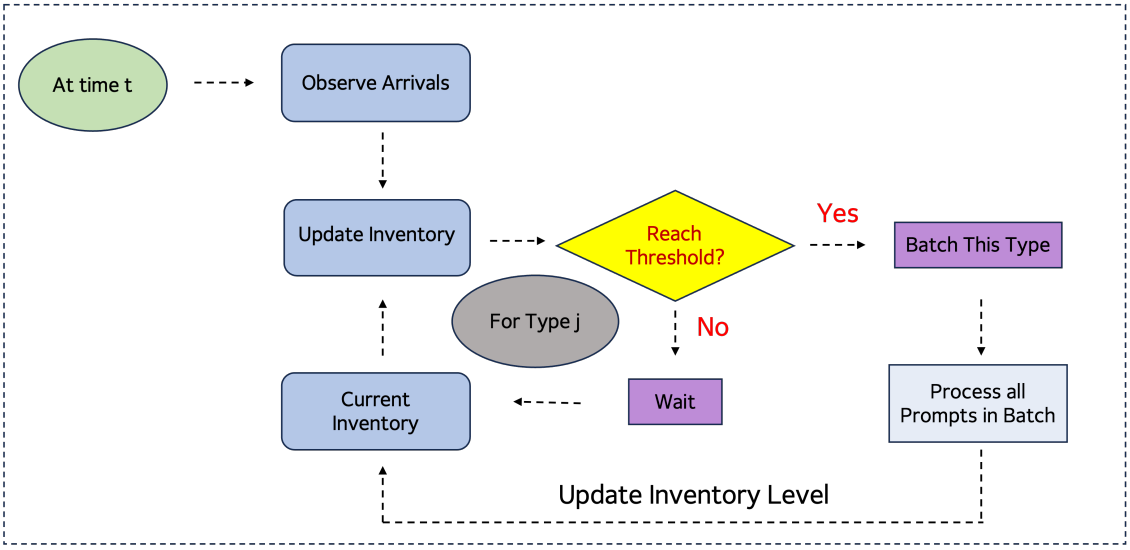

Figure 4 Pipeline of Algorithm 1

near equilibrium size M^*, treating memory as a resource to maximize throughput, not merely a constraint to satisfy.

Algorithm Description. For each prompt type $j \in [m]$, WAIT sets a threshold n_j derived from the fluid dynamics. At time t, the algorithm monitors the inventory n^t_{js}—the number of waiting prompts of type j at stage s. Type j prompts are included in a batch only when:

$$n^t_{j0} \geq n_j.$$

When this condition is met, WAIT constructs a batch by selecting $\min\{n_j, n^t_{js}\}$ prompts of type j at each stage s for all types meeting the threshold. If no type satisfies the condition, the algorithm waits for more arrivals. KV caches for waiting prompts are preserved, avoiding redundant computation. The complete specification is in Algorithm 1 and Figure 4.

4.3. Asymptotic Analysis

We evaluate the performance of WAIT in an asymptotic regime that captures long-run system behavior. Specifically, we scale the arrival rates to $\lambda^{(\zeta)}_j = \zeta\lambda_j$ and processing speeds accordingly $((d^{(\zeta)}_0, d^{(\zeta)}_1) = (\zeta^{-1}d_0, \zeta^{-1}d_1))$ while keeping memory capacity C fixed. This can equivalently be viewed as scaling the time horizon from T to ζT while keeping arrival rates and processing speeds unchanged. As $\zeta \to \infty$, we obtain infinite-horizon limits that characterize the system's long-run average performance, allowing us to study throughput stability under sustained load.

Algorithm 1 WAIT: Waiting for Accumulated Inference Threshold

Input: Memory C, arrival rates λ_j, thresholds n_j satisfying (10), $\forall j \in [m]$
Initialize prompt inventory $n_{js} \leftarrow 0$ for all $j \in [m], s \in \{0, 1, \ldots, l'_j\}$
Initialize event queue with arrival events for each type j
Set current time $t \leftarrow 0$
while True **do**
  Wait for the next event (arrival or batch completion)
  if event is an arrival of type j **then**
    Update inventory: $n_{j0} \leftarrow n_{j0} + 1$ ▷ Add new prompt to waiting queue of prefill phase
  else if event is a batch completion **then**
    Update inventory: For each j in batch B, $n_{js} \leftarrow n_{js} - \min\{n_j, n_{js}\}$ for $s = 0, \ldots, l'_j$
    Advance prompts: For each j, move $\min\{n_j, n_{js}\}$ prompts from stage s to $s+1$
    Clear KV caches for prompts reaching stage $l'_j + 1$ ▷ Free memory for completed prompts
  end if
  Check if $n_{j0} \geq n_j$ for any $j \in [m]$ ▷ Check threshold for batching
  if condition $n_{j0} \geq n_j$ is met for some j **then**
    Form batch B by selecting $\min\{n_j, n_{js}\}$ prompts of type j at each stage s for all j meeting the condition
    Process batch B, keep prompts outside the batch waiting and do not delete their KV caches
  end if
end while

Performance Metric: We focus on the average throughput, denoted $\textbf{Throughput}^{(\zeta,\pi)}$, where π represents the WAIT policy, defined as:

$$\textbf{Throughput}^{(\zeta,\pi)} = \frac{1}{\zeta T} \sum_{j=1}^{m} N_j^{(\zeta,\pi)},$$

with $N_j^{(\zeta,\pi)}$ being the total number of type-j tokens processed under scaling ζ. The average latency $\textbf{Latency}^{(\zeta,\pi)}$, $\textbf{TTFT}^{(\zeta,\pi)}$ and TTFT are defined similarly by dividing the quantity by ζ. Our objective is to demonstrate that $\textbf{Throughput}^{(\zeta,\pi)}$ approaches the fluid equilibrium benchmark $\textbf{Throughput}^* = \sum_{j=1}^{m} \lambda_j l'_j$ (9) as $\zeta \to \infty$ while giving bound to the latency and TTFT.

4.4. Theoretical Analysis

When memory capacity satisfies $C \geq M^*$ (where M^* is the equilibrium memory from Equation (7)), consider the WAIT policy π parameterized by thresholds $[n_1, \ldots, n_m]$, where n_j is the threshold for type-j prompts at each stage. WAIT is asymptotically optimal when the thresholds satisfy:

$$\begin{aligned} \Delta T(n_{1:m}) &:= d_0 + d_1 M^\pi \leq \tfrac{n_j}{\lambda_j}, \ \forall j \in [m], \\ M^\pi &= \sum_{j=1}^{m} n_j (l'_j + 1)\left(l_j + \tfrac{l'_j}{2}\right) \end{aligned} \tag{10}$$

The intuition follows from Equation (8): when type j is included in a batch, the iteration time ΔT ensures that expected arrivals of type j do not exceed n_j. This keeps the system close to the fluid equilibrium, using memory capacity fully for throughput while avoiding overflow.

THEOREM 1. *Assume the thresholds $\pi = [n_1, \ldots, n_m]$ satisfy (10), then we have*

$$\boldsymbol{Throughput}^* - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O((\zeta T)^{-\frac{1}{2}}),$$
$$\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O\left((\zeta T)^{\frac{1}{2}}\right).$$

Moreover, when we have $\Delta T_{[1,\ldots,m]} < n_j/\lambda_j, \forall j \in [m]$ where $\Delta T_{[1,\ldots,m]}$ is defined in (10), it holds that

$$\boldsymbol{Throughput}^* - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O(1/(\zeta T)),$$
$$\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O(1).$$

WAIT prevents eviction cascades by requiring $n^t_{j0} \geq n_j$ before admitting type-j prompts, ensuring arrivals do not exceed completions and keeping each type near equilibrium (avoiding FCFS failures in Example 2). Analytically, the threshold *decouples* the m types: we analyze each independently, as the fixed threshold breaks inter-type memory dependencies that would otherwise resist standard queueing analysis. With decoupling, we apply standard techniques (coupling, drift analysis) to establish asymptotic optimality. Complete proof in Appendix EC.2.

Theorem 1 provides performance bounds that match the worst-case lower bounds for latency and Time to First Token (TTFT). The following proposition establishes these lower bounds.

PROPOSITION 4. *There exists an instance where $C = M^*$, and for any non-predictive online policy π, the expected average latency and TTFT satisfy*

$$\mathbb{E}[\boldsymbol{Latency}^{(\zeta,\pi)}], \quad \mathbb{E}[\boldsymbol{TTFT}^{(\zeta,\pi)}] = \Omega((\zeta T)^{1/2}).$$

Moreover, we demonstrate that the latency for type j prompts depends exclusively on the relationship between $\Delta T_{[1,\ldots,m]}$ and n_j/λ_j. Specifically, when $\Delta T_{[1,\ldots,m]} < n_j/\lambda_j$, the average latency and TTFT for type j prompts are $O(1)$, offering a more precise performance estimate than that provided in Theorem 1.

A limitation of WAIT is its reliance on known arrival types and rates, which may not hold in realistic environments (though we can approximate it by using predictors, see e.g. (Fu et al. 2025)). In the next section, we address this issue by extending WAIT to handle unknown arrival types, introducing a nested variant that adapts to stochastic settings with unknown prompt types upon arrival.

5. Scheduling with Unknown Arrival Types: The Nested WAIT Algorithm

WAIT achieves near-optimal throughput when prompt types are known at arrival. In practice, however, output lengths are unknown and reveal gradually during decode. The *Nested WAIT* algorithm exploits this: **prompts classify themselves on-the-fly** as they decode, with short prompts completing early and longer prompts advancing to later segments. **No prediction is needed**.

EXAMPLE 3 (UNKNOWN OUTPUT LENGTHS). Continuing Example 2, suppose output lengths are now *unknown* at arrival: each "Hello" prompt receives either "Hi" (1 token) or "Hi there!" (2 tokens). The scheduler faces a dilemma. If it optimistically assumes all outputs are short (1 token each), it admits 6 prompts with capacity $C = 12$, requiring $6 \times 2 = 12$ units after decode. But if some outputs turn out to be 2 tokens (requiring 3 units each), memory overflows and eviction cascades begin. Conversely, if the scheduler conservatively assumes all outputs are long, it admits only 4 prompts to stay safe, wasting 25% of capacity when most outputs are actually short.

This dilemma—optimism leads to eviction, pessimism wastes capacity—leads to our design principles: (i) *avoid eviction* by not over-committing memory; (ii) *classify on-the-fly* by letting prompts reveal their type as they decode; (iii) *don't waste throughput* by exploiting available capacity; (iv) *let shorter prompts finish first* so they free memory without being blocked by longer ones.

5.1. Algorithm Overview

Nested WAIT uses **nested thresholds** to resolve the dilemma in Example 3: segment k handles prompts from stage $l'_{k-1} + 1$ to l'_k (with $l'_0 = 0$), each with threshold n_k. Short prompts complete in early segments without being blocked by long prompts; each segment operates at high memory utilization for its remaining types.

Formally, let output lengths satisfy $l'_1 < l'_2 < \cdots < l'_m$, where type j has decode length l'_j. We organize inference into m nested segments, where segment k spans decode stages from $l'_{k-1} + 1$ to l'_k (with $l'_0 = 0$). The algorithm uses thresholds $\pi = [n_1, \ldots, n_m]$ where n_k controls when segment k begins processing and limits batch size per stage.

Segment 1 processes all newly arrived prompts (at combined rate $\lambda_1 + \cdots + \lambda_m$) through decode stages $0, 1, \ldots, l'_1$. After $l'_1 + 1$ iterations, two outcomes emerge: type-1 prompts have completed their output and **exit the system**, freeing their memory; all other prompts (types 2 through m) **advance to segment 2**. Segment 2 now processes only these longer prompts (at reduced rate $\lambda_2 + \cdots + \lambda_m$)

through stages $l_1' + 1, \ldots, l_2'$. This pattern repeats: after $l_2' - l_1'$ more iterations, type-2 prompts exit, and types 3 through m advance to segment 3. **Prompts classify themselves** by completing (short) or continuing (long) at each segment boundary.

If segment k^* has fewer than n_{k^*} prompts, we pause processing for that segment and all later ones, retaining their KV caches on GPU. We continue batching exactly n_k prompts per stage in segments $k < k^*$ and perform iterations. Since new long prompts arrive continuously, segment k^* resumes once its threshold is met. Algorithm 2 formalizes this procedure, and Figure 5 illustrates the pipeline.

We assume identical prefill lengths $l_1 = l_2 = \cdots = l_m$ for clarity, though the general case extends similarly to Algorithm 1 by forming segment groups for each prefill length. The number of output lengths m minimally impacts performance, which depends primarily on total arrival rates. The number of segments can also differ from m while retaining near-optimal guarantees (Section 7).

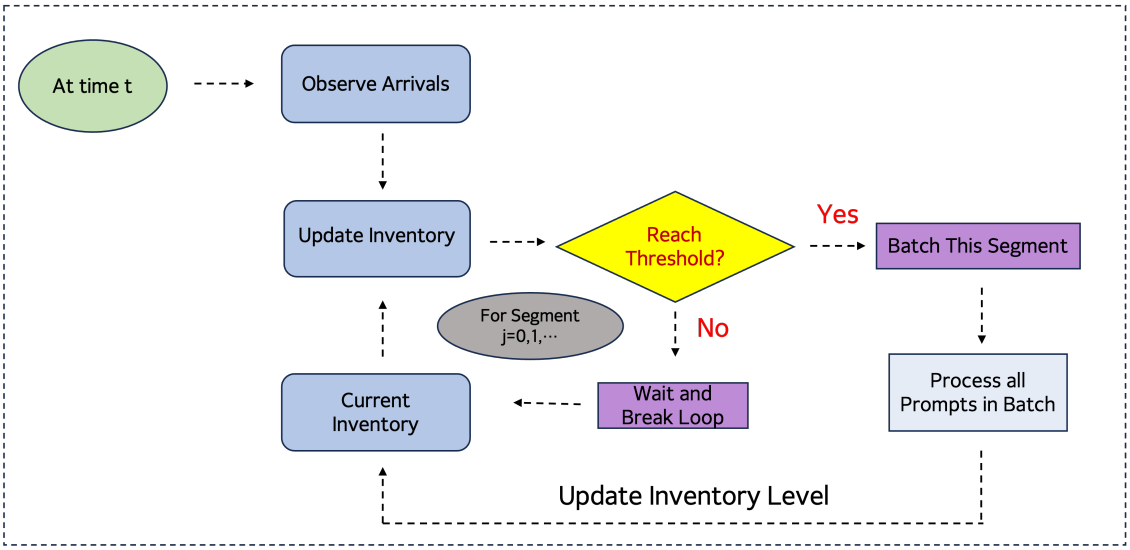

Figure 5 Pipeline of the Nested WAIT Algorithm

Consider Example 3 with two prompt types: "Hello" → "Hi" (1 token, type 1) and "Hello" → "Hi there!" (2 tokens, type 2). At arrival, we cannot distinguish them. After one decode iteration, however, **the prompts reveal themselves**: type-1 prompts have completed their output ("Hi") and exit; type-2 prompts have generated only their first token ("Hi") and continue. Nested WAIT exploits this revelation by partitioning processing into two segments. **Segment 1** processes all newly arrived prompts (both types) through stages 0, 1 (up to $l_1' = 1$) at combined rate $\lambda_1 + \lambda_2$. **Segment 2** processes

only type-2 prompts (those that survived segment 1) through stage 2 (up to $l_2' = 2$) at reduced rate λ_2. **No prediction is needed**—prompts simply classify themselves by completing or continuing at the segment 1 boundary.

Figure 6 visualizes the algorithm in action. At time $t = 0$, prompts $P1$ and $P2$ arrive but are insufficient to trigger processing, as Segment 1 has fewer than 3 prompts. At $t = 1$, $P3$ arrives, satisfying the Segment 1 threshold, and the batch is processed. Since these prompts have just begun decoding, none qualify for Segment 2. At $t = 5$, a new wave of prompts ($P4$, $P5$) arrives, again below the threshold. By $t = 7$, the threshold is met with $P6$, $P7$, and processing proceeds. Now, $P3$, having reached a deeper decode stage, transitions to Segment 2. However, since the Segment 2 threshold is still unmet, only Segment 1 is processed. Finally, at $t = 14$, enough prompts have advanced into Segment 2 ($P3$, $P5$, $P6$, etc.) to satisfy both segment thresholds. At this point, the Nested WAIT algorithm jointly schedules both segments, batching according to type-specific readiness. This example shows how the algorithm defers processing to ensure type separation and stage-aligned batching.

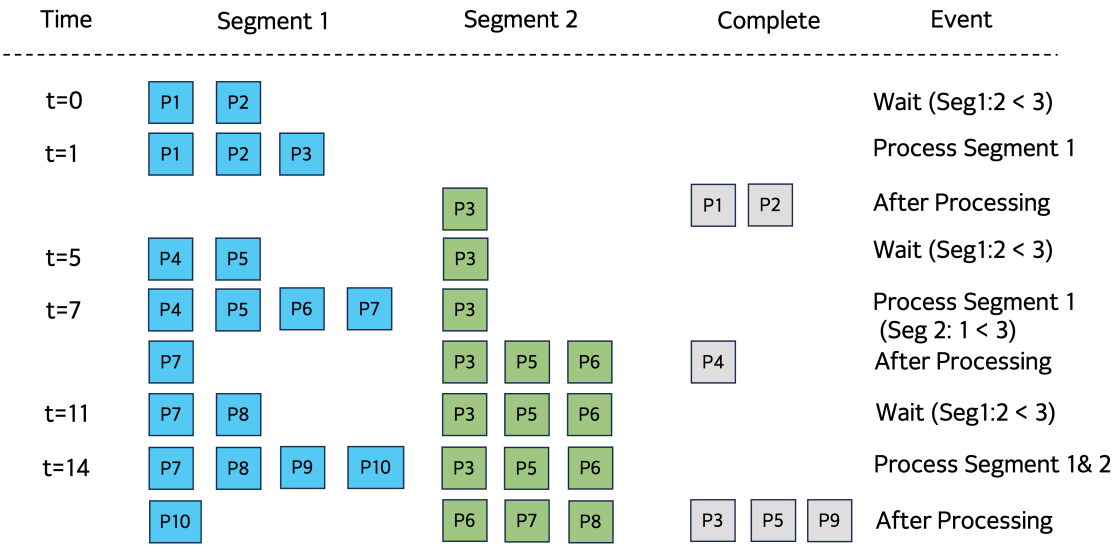

Figure 6 Example of the Nested WAIT Algorithm

This nested structure directly implements the four design principles from Example 3: (i) we **avoid eviction** by never exceeding thresholds; (ii) prompts **classify on-the-fly** at segment boundaries; (iii) we **don't waste throughput** because each segment operates at its own optimal threshold; (iv) **short prompts finish first** within segment 1, freeing memory without waiting for long prompts. Crucially,

Algorithm 2 Nested WAIT: Nested Waiting for Accumulated Inference Threshold

Input: Memory capacity C, arrival rates λ_j for $j \in [m]$, thresholds n_k for segments $k \in [m]$
Input: Output lengths $l'_1 < l'_2 < \cdots < l'_m$ for prompt types $j \in [m]$
▷ $Q_{k,s}$: number of prompts in segment k at decode stage s; entry stage of segment k is l'_{k-1} (with $l'_0 = 0$)
Initialize $Q_{k,s} \leftarrow 0$ for all segments $k \in [m]$ and stages $s \in [l'_{k-1}, l'_k - 1]$
Initialize event queue; set current time $t \leftarrow 0$
while True **do**
  Wait for the next event (arrival or batch completion)
  if event is an arrival **then**
    $Q_{1,0} \leftarrow Q_{1,0} + 1$ ▷ New prompts enter segment 1 at stage 0
  else if event is a batch completion for segment k **then**
    for each stage s in segment k **do**
      Advance $\min\{n_k, Q_{k,s}\}$ prompts from stage s to $s+1$
    end for
    if prompts reach stage l'_k (end of segment k) **then**
      if $k < m$ **then**
        Move to segment $k+1$ at stage l'_k ▷ These are type-$(k+1)$ or longer
      else
        Complete and clear KV caches
      end if
    end if
  end if
  Find largest k such that $Q_{k',l'_{k'-1}} \geq n_{k'}$ for all $k' \leq k$ ▷ All segments up to k meet threshold
  if such k exists **then**
    Form batch from segments $1, \ldots, k$: select $\min\{n_{k'}, Q_{k',s}\}$ prompts per stage
    Process batch; prompts outside batch wait with KV caches retained
  end if
end while

no prediction is needed—prompts simply reveal their output by completing or continuing. This achieves high memory utilization **for all types simultaneously**, not just the shortest or longest. We now formalize this intuition.

5.2. Main Theorem

We now analyze Nested WAIT in the asymptotic regime, following the same framework as Section 4. The unknown-output-length setting introduces **additional stochastic variability**: since we cannot predict which prompts will be long, some may queue at segment boundaries waiting for thresholds. This affects both memory requirements and throughput guarantees. We characterize average throughput $\mathbf{Throughput}^{(\zeta,\pi)}$, latency $\mathbf{Latency}^{(\zeta,\pi)}$, and time-to-first-token $\mathbf{TTFT}^{(\zeta,\pi)}$ under threshold policy $\pi = [n_1, \ldots, n_m]$.

Before showing Nested WAIT achieves near-optimal throughput, we establish a lower bound showing that **some** extra memory beyond M^* is necessary when output lengths are unknown.

Intuition: When memory capacity equals exactly $C = M^*$ (the fluid equilibrium), any algorithm that cannot predict output lengths faces a dilemma at every iteration. If we admit too many prompts

optimistically, some will reveal longer outputs than expected, causing overflow and eviction. If we admit too few conservatively, we waste capacity. Either scenario creates a throughput gap of $\Omega(1)$ over time.

PROPOSITION 5. *When memory capacity C equals the equilibrium memory M^*, there exists an instance where any online non-predictive policy π, unaware of prompt types upon arrival, incurs a throughput gap* $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^{(\zeta,\pi)}] = \Omega(1)$.

This lower bound shows that uncertainty has a cost: we need more memory than the fluid equilibrium to hedge against unknown output lengths. Theorem 2 shows that only a logarithmic amount of extra memory suffices.

As in Section 4, thresholds must balance two goals: (i) **throughput efficiency** (don't wait too long to form batches); (ii) **segment stability** (ensure later segments receive enough prompts). This leads to two constraints, analogous to Equation (10).

Similar to Equation (10), we define $\Delta T_{[1,\ldots,m]}(n_1,\ldots,n_m)$ as the time cost per iteration with exactly n_k prompts per stage in segment k, for all $k \in [1,\ldots,m]$. Thresholds must satisfy:

$$\begin{aligned} &\Delta T_{[1,\ldots,m]}(n_1,\ldots,n_m) < \frac{n_1}{\sum_{j=1}^{m}\lambda_j}, \\ &\frac{n_{k+1}}{n_k} > p_k, \forall k \in \{1,\ldots,m-1\}, \end{aligned} \tag{11}$$

where $p_k = (\sum_{j=k+1}^{m}\lambda_j)/(\sum_{j=k}^{m}\lambda_j) < 1$. To state the memory requirement, we define the memory usage when running iteration with exactly n_k prompts per stage in segment k, for all $k \in [1,\ldots,m]$ as:

$$\begin{aligned} M^{\pi} &= \sum_{k=1}^{m} n_k \left(l + \tfrac{1}{2}L'_k\right)\Delta l'_k, \\ \text{where } L'_k &= \sum_{r=1}^{k} l'_r,\ \Delta l'_k = l'_k - l'_{k-1},\ l'_0 = 0. \end{aligned} \tag{12}$$

Here, the formula captures the peak batch memory when exactly n_k prompts occupy each segment k: the term n_k is the threshold (maximum prompts concurrently decoding in segment k), $\Delta l'_k = l'_k - l'_{k-1}$ counts the decode stages spanned by segment k, and $(l + \frac{1}{2}L'_k)$ represents the average memory per prompt within segment k, where $L'_k = l'_1 + \cdots + l'_k$ is the cumulative decode length. We define the total number of iterations over time horizon $[0,T]$ as B. Therefore, we have $T \geq \sum_{b=1}^{B}\Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{b\text{-th Batch}}$ denotes the time cost of the b-th iteration

and $\Delta T_{\min} = \min_b\{\Delta T_{\text{Batch}}\}$. Theorem 2 establishes asymptotic optimality for Nested WAIT in the asymptotic regime with unknown prompt types, given thresholds $\pi = [n_1, \ldots, n_m]$ satisfying Equation (11).

THEOREM 2. *Assume thresholds $\pi = [n_1, \ldots, n_m]$ satisfy Equation (11), and memory M satisfies:*

$$\begin{aligned} M^{(\zeta,\pi)} &\geq M^{\pi} + \sum_{k=2}^{m} (l + l'_{k-1}) n_k \\ &\quad + \sum_{k=2}^{m} (l + l'_{k-1}) \theta_k^{-1} \ln\left(\frac{m\zeta B}{\delta}\right) \\ &= O\left(2M^{\pi} + \sum_{k=1}^{m} (l + l'_{k-1}) \theta_k^{-1} \ln\left(\frac{m\zeta B}{\delta}\right)\right). \end{aligned} \tag{13}$$

where $\theta_k \geq 8(n_k - n_{k-1}p_{k-1})/n_{k-1}$ (with $p_0 = 1$). Then:

$$\begin{aligned} &\boldsymbol{Throughput}^* - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O\left((\zeta T)^{-1}\right), \\ &\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ during time horizon $[0, T]$.

Nested WAIT extends WAIT's dimensionality reduction to unknown output lengths. Thresholds n_k bound each segment's state space, while prompts classify themselves on-the-fly by completing or continuing at segment boundaries—no prediction needed. The key technical challenge: prompts entering segment $k+1$ are those surviving segment k, creating a delayed arrival process. We address this via coupling: transitions from segment k to $k+1$ couple to a Poisson process with rate $\sum_{j=k+1}^{m} \lambda_j$, enabling independent per-segment analysis.

This coupling explains the **safety buffer** in Equation (13): when segment k has fewer than n_k prompts, queueing at boundaries introduces stochastic fluctuations. Doob's inequality bounds maximum queue length with high probability as $O(\ln(T/\delta))$—this logarithmic overhead is the safety buffer. Despite unknown output lengths, thresholds keep segments near equilibrium, preventing cascading evictions (Example 2). Full proof in Appendix EC.3.

The memory requirement in Equation (13) consists of two components with clear operational interpretations:

1. **Base memory** M^π: This is the equilibrium memory from the fluid model—the memory needed when exactly n_k prompts occupy each stage in segment k. This represents the "steady-state" memory consumption.

2. **Safety buffer** $\sum_{k=2}^{m}(l+l'_{k-1})(n_k+\theta_k^{-1}\ln(m\zeta B/\delta))$: This additional memory hedges against stochastic fluctuations when output lengths are unknown. It has three components:

- n_k: Accounts for prompts queueing at segment k boundaries, waiting for thresholds.
- $\theta_k^{-1}\ln(m\zeta B/\delta)$: High-probability protection against queue buildup (overflow occurs with probability $\leq \delta$).
- $(l+l'_{k-1})$: Memory footprint of prompts at the boundary between segments $k-1$ and k.

The safety buffer is *logarithmic* in T and $1/\delta$, making it modest compared to base memory M^π (Proposition 5 shows some extra memory is necessary; Theorem 2 shows $O(\ln(T/\delta))$ suffices). Performance depends primarily on arrival rates and output lengths, not m. In practice, we use a **segment-wise variant** grouping m types into $L \leq m$ segments (Section 7), addressing robustness to varying distributions and infeasibility of per-type thresholds for long decode lengths. Section 6 provides numerical validation.

6. Numerical Experiments

We evaluate (Nested) WAIT using Microsoft Vidur (simulating NVIDIA A100) (Agrawal et al. 2024a) on synthetic and real datasets Zheng et al. (2023). Benchmarks: vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023) (FCFS-based engines).

6.1. WAIT Algorithm (Known Output Lengths)

We test two scenarios with Poisson arrivals: (i) **Low demand**: $m=2$, $(l_1,l_2)=(10,10)$, $(l'_1,l'_2)=(10,20)$, $(\lambda_1,\lambda_2)=(1000,1000)$; (ii) **High demand**: $m=3$, $(l_1,l_2,l_3)=(20,20,20)$, $(l'_1,l'_2,l'_3)=(100,200,300)$, $(\lambda_1,\lambda_2,\lambda_3)=(6000,4000,2000)$. Thresholds: $n_j = B\cdot\rho_j/(l'_j+1)$, where $\rho_j = \lambda_j/\sum_{j'}\lambda_{j'}$. WAIT outperforms benchmarks (Figure 8).

Single-type workload. We test single-type scenarios with $(l_1,l'_1)=(630,20)$ to demonstrate stability region expansion (Figure 7). Without PD disaggregation (left), WAIT has higher latency at low rates due to threshold waiting, but at high rates baselines become unstable (latency $\to\infty$) while WAIT remains stable, achieving ~12% higher maximum throughput. With PD disaggregation (right), vLLM is unstable across *all* tested rates due to severe eviction cascades, while WAIT maintains both higher throughput (~40%) and lower latency. PD systems satisfy our linear model exactly (Zhong et al. 2024, Patel et al. 2024).

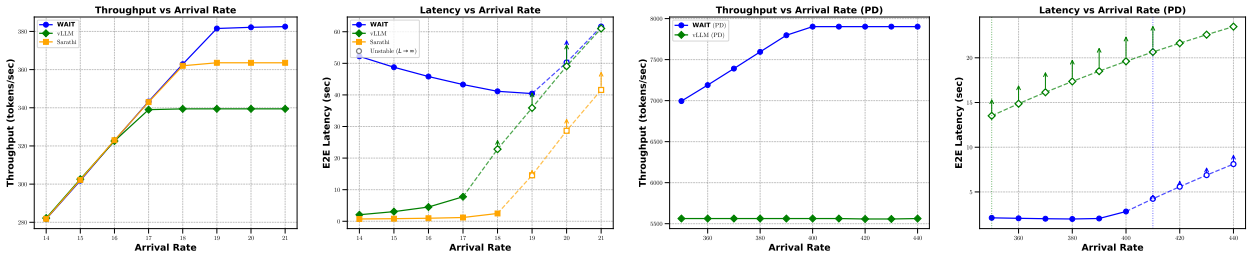

Figure 7 **WAIT on single-type workload. Left two: no PD disaggregation; Right two: with PD disaggregation. The number of requests tested for each data point is 1000.**

Multi-type workload (overloaded regime). Figure 8 shows time-series at fixed overloaded arrival rates. WAIT achieves ∼30% higher steady-state throughput than vLLM by preventing eviction cascades. Latency grows for all methods (confirming instability), but WAIT maintains lower growth rate.

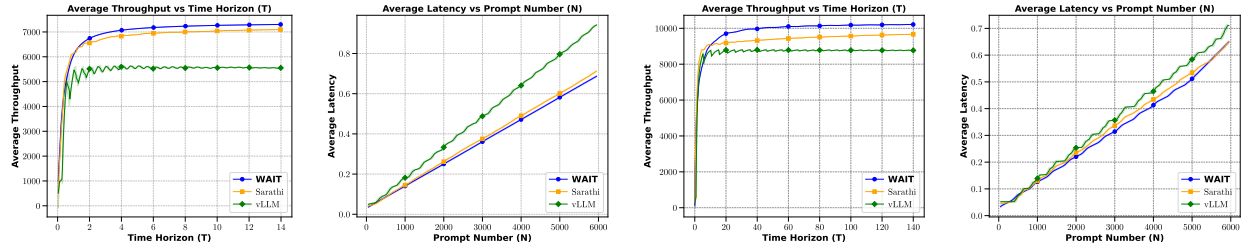

Figure 8 **Multi-type time-series. Left two: low demand; Right two: high demand.**

6.2. Nested WAIT (Unknown Output Lengths)

We test on synthetic data ($m = 4$ types, prefill=10, decode=20–160) and LMSYS-Chat-1M (Zheng et al. 2023) (50,000 prompts, $L = 10$ segments). Figure 9 shows time-series at overloaded rates: Nested WAIT maintains stable throughput while vLLM's declines due to eviction overhead. On-the-fly classification allows short prompts to complete quickly. Figure 10 confirms robustness on trace replay: Nested WAIT improves throughput by 15–30%.

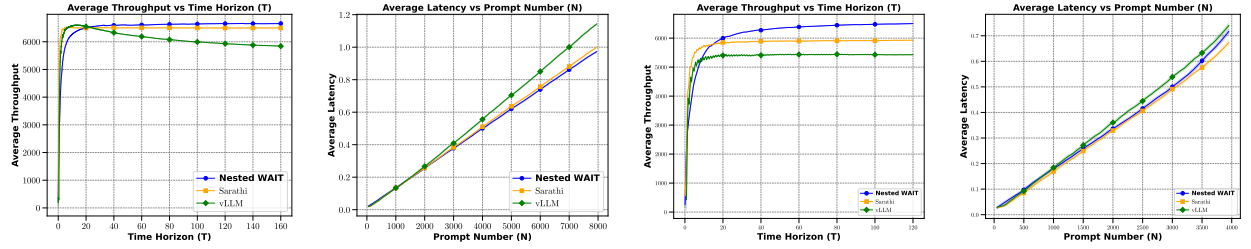

Figure 9 **Nested WAIT time-series. Left two: synthetic ($m = 4$); Right two: LMSYS (QPS=550).**

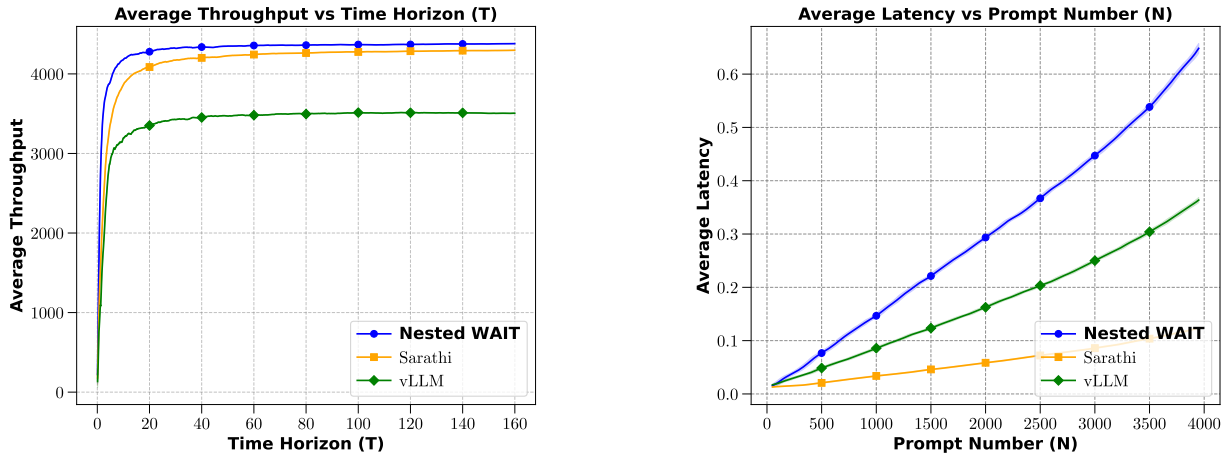

Figure 10 **Nested WAIT on LMSYS trace replay.**

7. Extensions

WAIT and Nested WAIT extend naturally in two directions: time-varying arrivals λ_j^t (using accumulated arrival constraints) and segment-wise design (grouping m types into $L \le m$ segments). Both extensions preserve asymptotic optimality.

7.1. Time-Varying Arrival Rates

For time-varying λ_j^t, replace constant rates with accumulated arrivals $\lambda_j[t_1,t_2] := \int_{t_1}^{t_2} \lambda_j^t \, dt$. Threshold conditions (11) generalize to: for all $t \in [0,T]$, $0 < \Delta t \le \Delta T$, and $k = 1,\dots,m-1$,

$$\sum_{j=1}^{m} \lambda_j[t,t+\Delta t] < n_1, \quad n_{k+1}/n_k > p_k[t,t+\Delta t], \tag{14}$$

where $p_k[t,t+\Delta t] = (\sum_{j=k+1}^{m} \lambda_j[t,t+\Delta t])/(\sum_{j=k}^{m} \lambda_j[t,t+\Delta t])$.

THEOREM 3. *Under these time-varying thresholds with safety buffer* $M^{(\zeta,\pi)} = O(2M^{\pi} + \sum_{k=1}^{m} \theta_k^{-1}(l + l'_{k-1}) \ln(m\zeta B/\delta))$, *Nested WAIT achieves* $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^{(\zeta,\pi)}] = O((\zeta T)^{-1})$, $\mathbb{E}[\boldsymbol{Latency}^{(\zeta,\pi)}] = O(1)$, *with overflow probability* $< \delta$.

The coupling construction extends naturally using time-varying Poisson processes. Proof in Appendix EC.4.

7.2. Segment Design

The **segment-wise design** groups m types into $L \le m$ segments (e.g., $L = 10$ for $m = 500$ in LMSYS Zheng et al. (2023)), reducing complexity from $O(m)$ to $O(L)$. Partition types with decode lengths $l'_1 < \cdots < l'_m$ into L segments with $\Delta L := m/L$ types each. Segment k has aggregate rate $\lambda'_k = \sum_{(k-1)\Delta L+1 \le j \le k\Delta L} \lambda_j$.

Given a policy $\pi = [n_1, \ldots, n_L]$ with threshold n_k for the k-th segment in Algorithm 2, we formulate a linear system analogous to Equation (11):

$$\begin{aligned} \Delta T_{[1:L]}(n_{1:L}) &\le \frac{n_1}{\sum_{r=1}^{L} \lambda'_r}, \\ \frac{n_{k+1}}{n_k} &> p_k, \ \forall k < L, \end{aligned} \tag{15}$$

where $p_k = \frac{\lambda'(k+1 \to L)}{\lambda'(k \to L)}$, $\Delta T_{[1,\ldots,L]}(n_1, \ldots, n_L) = d_0 + d_1 M^{\pi}(n_1, \ldots, n_L)$, and memory is:

$$M^{\pi}(n_1, \ldots, n_L) = \sum_{k=1}^{L} n_k \left(l + \frac{k}{2} \Delta L \right) \Delta L.$$

THEOREM 4. *If* $\pi = [n_1, \ldots, n_L]$ *satisfies* (15) *and* $M^{(\zeta,\pi)} = O(2M^{\pi} + \sum_{k=1}^{L}(l + l'_{k-1})\theta_k^{-1} \ln(m\zeta B/\delta))$, *then* $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^{(\zeta,\pi)}] = O((\zeta T)^{-1})$, $\mathbb{E}[\boldsymbol{Latency}^{(\zeta,\pi)}] = O(1)$, $\mathbb{E}[\boldsymbol{TTFT}^{(\zeta,\pi)}] = O(1)$, *with overflow probability* $< \delta$ *over* $[0, T]$.

Proof follows Theorem 2 in Appendix EC.3: aggregated rates preserve the Poisson coupling structure.

Figure 11 validates using LMSYS data ($m = 500$): total memory exhibits U-shape with minimum at $L = 5$-10, confirming $L = 10$ (Section 6).

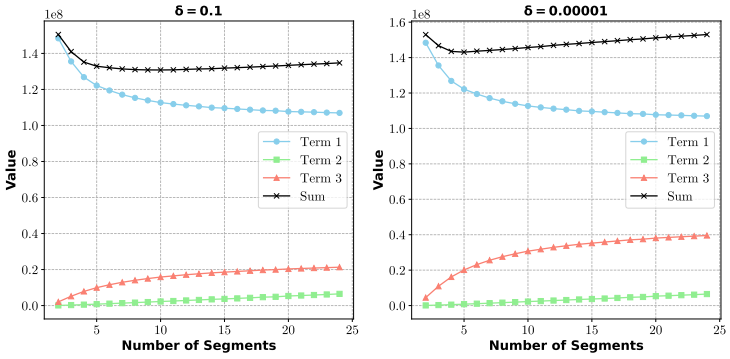

Figure 11 **Memory components vs. segments L (rate=500, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.**

8. Conclusion

We introduce the (Nested) WAIT algorithm, which provides theoretical guarantees for LLM inference scheduling under unknown output lengths. Using queueing theory and fluid models, our algorithms achieve 12–25% throughput improvement and 30% latency reduction compared to existing systems. Future directions include: (1) adapting to time-varying arrivals and demand surges

(Section 7 shows preliminary results for moderate variations), (2) extending to multi-GPU distributed systems with load balancing across facilities, and (3) integrating emerging optimizations such as multi-head latent attention (DeepSeek-AI 2024) into theoretical frameworks.

Endnotes

References

Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024a) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.

Agrawal A, Kedia N, Panwar A, Mohan J, Kwatra N, Gulavani B, Tumanov A, Ramjee R (2024b) Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–134.

Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* .

Anthropic (2023) Claude. https://claude.ai.

Asmussen S (2003) *Applied probability and queues*, volume 2 (Springer).

Balkanski E, Rubinstein A, Singer Y (2016) The power of optimization from samples. *Advances in Neural Information Processing Systems* 29.

Bäuerle N (2002) Optimal control of queueing networks: An approach via fluid models. *Advances in Applied Probability* 34(2):313–328.

Cole R, Roughgarden T (2014) The sample complexity of revenue maximization. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 243–252.

DeepSeek-AI (2024) Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. URL https://arxiv.org/abs/2405.04434.

Devanur NR, Hayes TP (2009) The adwords problem: online keyword matching with budgeted bidders under random permutations. *Proceedings of the 10th ACM conference on Electronic commerce*, 71–78.

Durrett R (2019) *Probability: theory and examples*, volume 49 (Cambridge university press).

Fu Y, Zhu S, Su R, Qiao A, Stoica I, Zhang H (2025) Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37:59006–59029.

García-Martín E, Rodrigues CF, Riley G, Grahn H (2019) Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing* 134:75–88.

GitHub (2022) Copilot. URL https://github.com/features/copilot.

Google (2023) Bard. https://bard.google.com.

Im S, Moseley B (2013) Online batch scheduling for flow objectives. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 102–104.

Kang H, Zhang Q, Kundu S, Jeong G, Liu Z, Krishna T, Zhao T (2024) Gear: An efficient kv cache compression recipefor near-lossless generative inference of llm. *arXiv e-prints* arXiv–2403.

Kingman JFC (1962) On queues in heavy traffic. *Journal of the Royal Statistical Society: Series B (Methodological)* 24(2):383–392.

Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez J, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.

Lattanzi S, Lavastida T, Moseley B, Vassilvitskii S (2020) Online scheduling via learned weights. *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1859–1877 (SIAM).

Li W, Wang L, Chai X, Yuan H (2020) Online batch scheduling of simple linear deteriorating jobs with incompatible families. *Mathematics* 8(2):170.

Li Y, Dai J, Peng T (2025) Throughput-optimal scheduling algorithms for llm inference and ai agents. URL https://arxiv.org/abs/2504.07347, accessed: 2025-04-11.

Liu P, Lu X (2015) Online unbounded batch scheduling on parallel machines with delivery times. *Journal of Combinatorial Optimization* 29:228–236.

Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations research* 59(4):835–846.

Lucier B, Menache I, Naor J, Yaniv J (2013) Efficient online scheduling for deadline-sensitive jobs. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 305–314.

Maglaras C (2000) Discrete-review policies for scheduling stochastic networks: Trajectory tracking and fluid-scale asymptotic optimality. *The Annals of Applied Probability* 10(3):897–929.

Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for markovian service networks. *Queueing Systems* 30(1):149–201.

OpenAI (2024) Gpt-4 technical report. URL https://arxiv.org/abs/2303.08774.

Patel D (2023) Peeling The Onion's Layers: Large Language Models' Cost, Context, and Feature Breakdown. Semianalysis blog, URL https://semianalysis.com/2023/02/13/peeling-the-onions-layers-large-language/, accessed: [Insert Date Here].

Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.

Pope R, Douglas S, Chowdhery A, Devlin J, Bradbury J, Heek J, Xiao K, Agrawal S, Dean J (2023) Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5:606–624.

Strait P (1974) On the maximum and minimum of partial sums of random variables. *Pacific Journal of Mathematics* 52(2):585–593.

Vee E, Vassilvitskii S, Shanmugasundaram J (2010) Optimal online assignment with forecasts. *Proceedings of the 11th ACM conference on Electronic commerce*, 109–118.

Yu GI, Jeong JS, Kim GW, Kim S, Chun BG (2022) Orca: A distributed serving system for {Transformer-Based} generative models. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–538.

Zheng L, Chiang WL, Sheng Y, Li T, Zhuang S, Wu Z, Zhuang Y, Li Z, Lin Z, Xing EP, et al. (2023) Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998* .

Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.

Online Supplement

Appendix EC.1: Proofs of Propositions

EC.1.1. Proof of Proposition 1

Consider a system with only type-j prompts. In equilibrium with batch size n_j, prompts are uniformly distributed across stages, yielding memory $M_j = n_j(l_j + l'_j/2)$ and iteration time $\tau_j = d_0 + d_1 M_j$. During each iteration, expected arrivals are $\tau_j \lambda_j$ while completions are $n_j/(l'_j + 1)$. The arrival-completion gap satisfies:

$$\text{Arrivals}_j - \frac{n_j}{l'_j + 1} = d_0 \lambda_j + n_j \left[\lambda_j d_1 (l_j + l'_j/2) - \frac{1}{l'_j + 1} \right] \geq d_0 \lambda_j > 0,$$

where the inequality follows from $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$. This positive gap implies queue length grows as $\Omega(N)$ after N iterations, establishing instability.

EC.1.2. Proof of Proposition 2

Throughput is bounded by the total output tokens generated, which cannot exceed arrivals' output lengths. Over time horizon $[0, T]$: $\mathbb{E}[\textbf{Throughput}^{(\pi,T)}] \leq \sum_{j=1}^{m} \mathbb{E}[\text{Arrivals}_j^T/T] \cdot l'_j = \sum_{j=1}^{m} \lambda_j l'_j = \textbf{Throughput}^*$.

EC.1.3. Proof of Proposition 3

Proof Outline. We construct a two-type example where FCFS suffers from cascading evictions even when capacity equals equilibrium memory M^*, leading to $\Omega(1)$ throughput gap.

Consider two prompt types: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$.

Fluid Equilibrium. The equilibrium equations $d_0 + d_1(1.5n_1 + 2n_2) = n_1/2 = n_2/3$ yield:

$$K = \frac{d_0}{1 - 9d_1}, \quad n_1^* = 2K, \quad n_2^* = 3K, \quad M^* = 9K.$$

Setting $K = 1$ (without loss of generality), we have $n_1^* = 2$, $n_2^* = 3$, $M^* = 9$.

Equilibrium Batch Composition. In equilibrium, the batch contains:

- Type 1: 1 prompt at stage 0 (memory = 1), 1 prompt at stage 1 (memory = 2, completing)
- Type 2: 1 prompt at stage 0 (memory = 1), 1 at stage 1 (memory = 2), 1 at stage 2 (memory = 3, completing)

Overflow Probability. Let (X_1, X_2) denote arrivals during one iteration, where $X_j \sim \text{Poisson}(1)$. The combinations that do not cause memory overflow within two iterations are: $(0,0), (1,0), (0,1), (1,1), (2,0)$. All other combinations lead to overflow. The overflow probability is:

$$p = 1 - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{2e^2} = 1 - \frac{9}{2e^2} \approx 0.391.$$

Since overflow probability is bounded away from zero, FCFS experiences eviction with constant probability at each iteration. This leads to queue length growth $\Omega(t)$ and throughput gap $\Omega(1)$.

EC.1.4. Proof of Proposition 5

Using the same two-type example as Proposition 3, the key constraint is that prompt types cannot be distinguished until a type-2 prompt enters stage 2. At stages 0 and 1, both types appear identical to the scheduler.

For any admissible batching policy and batch configuration, one of the following must hold: (1) **Memory underutilization**: The batch leaves significant memory unused, causing arrivals to exceed completions; or (2) **Overflow risk**: The batch fully utilizes memory but faces constant probability $\rho > 0$ of overflow. Since throughput is counted only upon completion, any policy incurs a throughput gap of at least $\Omega(\rho)$ compared to $\textbf{Throughput}^*$.

Appendix EC.2: Proof of Theorem 1

Proof Outline. This proof establishes asymptotic optimality of the WAIT algorithm when output lengths are known. The structure proceeds in two parts:

1. **Single-type case**: We construct a coupled dominating process using the Lindley recursion and apply Kingman's bound for queue length expectations.
2. **Multiple-type case**: We extend to m types by applying the coupling construction type-by-type, leveraging that the fixed threshold n_j decouples inter-type dynamics.

Key techniques include: Lindley recursion for queue length bounds (Asmussen 2003, Proposition 6.3), coupling construction to dominate the original process, Kingman's bound for negative drift processes (Kingman 1962), and standard random walk results (Strait 1974).

Throughout this appendix, we use b to denote the *batch index* (the count of completed batch iterations), which is distinct from the stage index $s \in \{0, 1, \ldots, l'_j\}$ used in the main text for decode stages. We use t for continuous time and T (or B) for the total number of batches over the time horizon.

We first consider the single-type case, then extend to multiple types.

EC.2.0.1. Single-Type Case Consider a single prompt type with threshold n. For general scheduling algorithms, analyzing this system in continuous time is challenging: KV cache grows dynamically during decode, arrivals are stochastic, and eviction risks complicate the dynamics.

The key insight of the WAIT policy is that its *threshold mechanism achieves dimensionality reduction*. By waiting until exactly n prompts accumulate before initiating a batch, the algorithm enforces a fixed batch size, which ensures constant processing time ΔT per batch. This decouples the complex continuous-time dynamics into a tractable discrete-time process. This reduction—from a complex memory-constrained queueing system to a simple random walk analysis—is enabled by the algorithm design, not an inherent simplicity of the problem.

Specifically, let $\tilde{\lambda}$ denote the continuous-time arrival rate. The WAIT policy's fixed batch size implies that arrivals during each batch follow Poisson$(\tilde{\lambda} \cdot \Delta T)$. Defining $\lambda = \tilde{\lambda} \cdot \Delta T$ (expected arrivals per batch), we can analyze the system as an *embedded Markov chain* observed at batch completion times, indexed by $b \in \mathbb{N}$. Setting $n = \lambda$ corresponds to critical loading. The throughput loss arises from "stuck" iterations where insufficient prompts are available.

LEMMA 1 **(Queue Length and Stuck Time).** *Let* $W^{b+1} = W^b + X^b - \lambda \cdot \mathbf{1}\{W^b + X^b \geq \lambda\}$, *where* $X^b \sim Poisson(\lambda)$, $W^0 = 0$. *Define* B_{stuck} *as stuck iterations. Then* $\mathbb{E}[W^B] = \lambda \cdot \mathbb{E}[B_{\text{stuck}}]$ *(proof in Appendix EC.5.1).*

LEMMA 2 **(Coupled Dominating Process).** *Define* $\tilde{W}^0 = 2\lambda$, $\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\}$. *Then* $\tilde{W}^b \geq W^b + \lambda$ *for all* $b \in \mathbb{N}$ *(proof in Appendix EC.5.2).*

Throughput gap analysis. By Lindley recursion (Asmussen 2003, Proposition 6.3), $\tilde{W}^B = 2\lambda + \max_{1 \leq k \leq B}(\sum_{r=1}^{k}(X^r - \lambda))^+$. Since $\tilde{W}^B \geq W^B + \lambda$, we bound $\mathbb{E}[W^B]$. By Strait (1974), $\mathbb{E}[\max_{1 \leq k \leq B}(S^k)^+] = \sum_{k=1}^{B} \frac{1}{k}\mathbb{E}[(S^k)^+]$. Using Cauchy-Schwarz and $\text{Var}(S^k) = \lambda k$, we obtain $\mathbb{E}[(S^k)^+] \leq \sqrt{\lambda k}$, yielding $\mathbb{E}[W^B] \sim \sqrt{\lambda B}$ as $B \to \infty$.

Asymptotic Regime. In the asymptotic regime where the time horizon scales as $\zeta \to \infty$:

$$\frac{1}{B^\zeta}\mathbb{E}[W^{B,\zeta}] = \frac{\lambda^\zeta}{B^\zeta} + \frac{(\lambda B)^{\zeta/2}}{B^\zeta} = \left(\frac{\lambda}{B}\right)^\zeta + \lambda^{\zeta/2}B^{-\zeta/2} \to 0 \quad \text{as } \zeta \to \infty.$$

EC.2.0.2. Multiple-Type Case We now extend to m prompt types. Consider the WAIT policy $\pi = [n_1, \ldots, n_m]$, where n_j is the threshold for type-j prompts.

Policy Class Π. Define the processing time for a full batch (containing all m types):

$$\Delta T_{[1,\ldots,m]} = d_0 + d_1 \cdot \sum_{j=1}^{m} n_j(l'_j + 1)\left(l_j + \frac{l'_j}{2}\right). \tag{16}$$

Approaching Load Balance. The constraint $\Delta T_{[1,\ldots,m]} \le n_j/\lambda_j$ (Equation 17) embodies the "approaching load balance" principle. For each type j, expected arrivals during a full batch equal $\lambda_j \cdot \Delta T_{[1,\ldots,m]}$. The constraint ensures arrivals do not exceed batch size n_j, preventing queue accumulation and eviction risk. The equilibrium policy π^* achieves equality (arrivals = completions), maximizing throughput while maintaining stability.

The policy parameters must satisfy:

$$\begin{aligned} &\Delta T_{[1,\ldots,m]} \le \frac{n_j}{\lambda_j} \text{ for all } j \in [m], \\ &M^\pi = \sum_{j=1}^{m} n_j(l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) \ge M^*. \end{aligned} \tag{17}$$

These constraints ensure that for any $\pi \in \Pi$: when all m types are batched together, arrivals do not exceed completions for each type (i.e., $\text{Arrival}_j \le \text{Completion}_j$). The equilibrium policy corresponds to $\pi^* = [\frac{n_1^*}{l'_1+1}, \cdots, \frac{n_m^*}{l'_m+1}]$ when $M^\pi = M^*$.

State transition. Let b denote batch index, J^b types in batch b. Queue dynamics: $W_{(j)}^{t_e(b+1)} = W_{(j)}^{t_e(b)} + X_{(j)} + Y_{(j)} - n_j \cdot \mathbf{1}\{j \in J^{b+1}\}$, where arrivals follow Poisson processes during waiting and processing. Fixed thresholds ensure constant full-batch time $\Delta T_{[1,\ldots,m]}$ from (16).

Type Decoupling. Fixed thresholds n_j enable type decoupling. Each type j has independent arrivals during batch processing: $X_{(j)}^b \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1,\ldots,m]})$. Arrivals depend only on the type-specific rate λ_j and batch time $\Delta T_{[1,\ldots,m]}$, not on other types' queue states. This independence allows us to analyze each type separately and sum the resulting bounds.

LEMMA 3 **(Multiple-Type Coupled Process)**. *Define a coupled process* $\{\tilde{W}_{(j)}^b\}$ *indexed by batch* b*:*

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + X_{(j)}^b - n_j\}, \quad X_{(j)}^b \sim \textit{Poisson}(\lambda_j \cdot \Delta T_{[1,\ldots,m]}).$$

Then $\tilde{W}_{(j)}^b \ge W_{(j)}^b$ *for all* $b \ge 0$ *and* $j = 1, \ldots, m$.

The proof is in Appendix EC.5.3. Define $Y_{(j)}^b = X_{(j)}^b - n_j$. The coupled process becomes:

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + Y_{(j)}^b\}.$$

Case 1: Zero Drift ($\Delta T_{[1,\ldots,m]} = n_j/\lambda_j$). When $\lambda_j \cdot \Delta T_{[1,\ldots,m]} = n_j$, we have $\mathbb{E}\left[Y_{(j)}^b\right] = 0$ (zero drift). By Lemma 1:

$$\mathbb{E}\left[W_{(j)}^B\right] \le \mathbb{E}\left[\tilde{W}_{(j)}^B\right] \le \lambda_j + \sqrt{\lambda_j B}.$$

Case 2: Negative Drift ($\Delta T_{[1,\dots,m]} < n_j/\lambda_j$). When $\mathbb{E}\left[Y^b_{(j)}\right] = \lambda_j \Delta T_{[1,\dots,m]} - n_j < 0$, the coupled process has negative drift. Since

$$\mathrm{Var}(Y^b_{(j)}) = \lambda_j \cdot \Delta T_{[1,\dots,m]}, \quad \left|\mathbb{E}\left[Y^b_{(j)}\right]\right| = n_j - \lambda_j \cdot \Delta T_{[1,\dots,m]},$$

by Kingman's bound (Kingman 1962):

$$\mathbb{E}\left[W^B_{(j)}\right] \le \mathbb{E}\left[\tilde{W}^B_{(j)}\right] \le 2n_j + \frac{\lambda_j \cdot \Delta T_{[1,\dots,m]}}{2\left(n_j - \lambda_j \cdot \Delta T_{[1,\dots,m]}\right)}.$$

Equilibrium Case ($C = M^*$). When $\pi^* = [\frac{n^*_1}{l'_1+1}, \cdots, \frac{n^*_m}{l'_m+1}]$, all types satisfy $\Delta T_{[1,\dots,m]} = n_j/\lambda_j$ (zero drift). Summing over all types:

$$\frac{1}{B}\sum_{j=1}^{m} \mathbb{E}\left[W^B_{(j)}\right] \le \frac{1}{B}\sum_{j=1}^{m} \lambda_j + \sum_{j=1}^{m}\sqrt{\frac{\lambda_j}{B}} = O(B^{-1/2}).$$

Asymptotic Regime. As $\zeta \to \infty$:

$$\frac{1}{\zeta B}\sum_{j=1}^{m} \lambda_j + \sum_{j=1}^{m}\sqrt{\frac{\lambda_j}{\zeta B}} \to 0.$$

The expected end-to-end latency scales as $O(B^{-1/2})$.

Strictly Negative Drift Case. When $\Delta T_{[1,\dots,m]} < n_j/\lambda_j$ for all j, the throughput gap is $O(B^{-1})$:

$$\frac{1}{B}\sum_{j=1}^{m} \mathbb{E}\left[W^B_{(j)}\right] \le \frac{1}{B}\sum_{j=1}^{m}\left(2n_j + \frac{\lambda_j \cdot \Delta T_{[1,\dots,m]}}{2\left(n_j - \lambda_j \cdot \Delta T_{[1,\dots,m]}\right)}\right) = O(B^{-1}).$$

Appendix EC.3: Proof of Theorem 2

Proof Outline. This proof establishes asymptotic optimality of Nested WAIT when output lengths are unknown. The algorithm achieves *on-the-fly classification*: prompts classify themselves by their actual output lengths at segment boundaries, requiring no prediction. The structure proceeds as follows:

1. **First segment analysis**: All prompts enter segment 1 with Poisson arrivals. We couple with a Lindley process and apply Kingman's inequality for bounded expectations.

2. **Subsequent segments analysis**: After completing segment $k-1$, prompts naturally reveal whether they continue to segment k. This type revelation follows Binomial thinning: a fraction p_k of completions continue, while short prompts finish without being delayed by longer ones. We apply Lindley coupling to each segment.

3. **High-probability bounds prevent eviction**: Expected bounds ensure average stability, but stochastic fluctuations risk overflow. We construct an exponential martingale and apply Doob's inequality to obtain high-probability memory bounds, preventing eviction cascades.

We adopt notation consistent with the main text (Section 5). Let l denote the common input length (tokens after prefill), and let l'_j denote the output length for type-j prompts. We use b to denote the *batch index* (count of completed iterations) and $k \in \{1,\dots,m\}$ to denote the *segment index*, where segment k contains prompts with output lengths in $(l'_{k-1}, l'_k]$. The threshold for segment k is n_k (from Equation (11)). We denote by $W^b_{(k)}$ the queue length for segment k after batch b, and by B the total number of batches over the time horizon. The thinning probability $p_k = (\sum_{j=k}^{m} \lambda_j)/(\sum_{j=k-1}^{m} \lambda_j)$ represents the fraction of prompts completing segment $k-1$ that continue to segment k.

EC.3.1. First Segment Analysis

All prompts initially enter segment 1 with Poisson arrivals at aggregate rate $\sum_{j=1}^{m} \lambda_j$. We construct a dominating process and apply Kingman's inequality to bound expected queue length.

For the first segment ($k = 1$), the state transition is:

$$W_{(1)}^{b+1} = W_{(1)}^{b} + Y_{(1)}^{b} - n_1 \cdot \mathbf{1}\{W_{(1)}^{b} + Y_{(1)}^{b} \geq n_1\}, \quad Y_{(1)}^{b} \sim \text{Poisson}(\Delta T_b \cdot \sum_{j=1}^{m} \lambda_j),$$

where ΔT_b denotes the processing time of batch b. Since $\Delta T_b \leq \Delta T_{[1,\ldots,m]}$ (the time for a full batch), we construct a dominating process:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} \geq n_1\}, \quad \bar{Y}_{(1)}^{b} \sim \text{Poisson}(\Delta T_{[1,\ldots,m]} \cdot \sum_{j=1}^{m} \lambda_j).$$

This is further dominated by a Lindley process:

$$\tilde{W}_{(1)}^{0} = 2n_1, \quad \tilde{W}_{(1)}^{b+1} = \max\{2n_1, \tilde{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} - n_1\}.$$

Since $\Delta T_{[1,\ldots,m]} \cdot \sum_{j=1}^{m} \lambda_j < n_1$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}\left[W_{(1)}^{b}\right] \leq 2n_1 + \frac{\Delta T_{[1,\ldots,m]} \cdot \sum_{j=1}^{m} \lambda_j}{2\left(n_1 - \Delta T_{[1,\ldots,m]} \cdot \sum_{j=1}^{m} \lambda_j\right)}.$$

Remark. Prompts waiting in the first segment are stored in CPU and do not consume GPU memory.

EC.3.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), the arrival process reflects on-the-fly classification: prompts that completed segment $k-1$ reveal whether their output lengths exceed l'_{k-1}. This type revelation follows binomial thinning with probability p_k.

Arrivals come from prompts that completed segment $k-1$:

$$Y_{(k)}^{b} \sim \text{Binomial}\left(n_{k-1},\, p_k\right), \quad \text{where } p_k = \frac{\sum_{j=k}^{m} \lambda_j}{\sum_{j=k-1}^{m} \lambda_j}.$$

On-the-fly Classification via Binomial Thinning. The arrival process $Y_{(k)}^{b} \sim \text{Binomial}(n_{k-1}, p_k)$ reflects the *on-the-fly classification mechanism*, not merely a modeling choice. After segment $k-1$ processing, exactly n_{k-1} prompts complete that stage. Among these, a fraction $p_k = \frac{\sum_{j=k}^{m} \lambda_j}{\sum_{j=k-1}^{m} \lambda_j}$ have output lengths exceeding l'_{k-1}. These prompts naturally continue to segment k. Prompts classify themselves by their actual output lengths—no prediction needed. Short prompts complete quickly without being delayed by longer ones.

The state transition is:

$$W_{(k)}^{b+1} = W_{(k)}^{b} + Y_{(k)}^{b} - n_k \cdot \mathbf{1}\{W_{(k)}^{b} + Y_{(k)}^{b} \geq n_k\},$$

where b indexes the batches containing segment $k-1$. Note that each segment has its own batch index, but all are bounded by the total batch count B.

Define $X_{(k)}^{b} = Y_{(k)}^{b} - n_k$. We construct a coupled dominating process:

LEMMA 4 (**Coupled Process for Segment** k). *Define* $\{\tilde{W}^b_{(k)}\}$ *by:*

$$\tilde{W}^0_{(k)} = n_k,$$
$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\}.$$

Then $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ *for all* $b \in \mathbb{N}$.

The proof is in Appendix EC.5.4. The process $\{\tilde{W}^b_{(k)}\}$ has the Lindley representation:

$$\tilde{W}^{b+1}_{(k)} = n_k + \max_{0\leq i\leq b}\{S^i_{(k)}\},$$
$$\text{where } S^i_{(k)} = \sum_{r=1}^{i} X^r_{(k)},\ S^0_{(k)} = 0.$$

Expected Queue Length. Since $\mathbb{E}\left[X^b_{(k)}\right] = n_{k-1}p_k - n_k < 0$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}\left[W^b_{(k)}\right] \leq \mathbb{E}\left[\tilde{W}^b_{(k)}\right] \leq n_k + \frac{n_{k-1}\cdot p_k(1-p_k)}{2\left(n_k - n_{k-1}\cdot p_k\right)}.$$

Negative drift ensures queue stability, preventing unbounded growth that would lead to eviction. This bound ensures finite expected queue length for each segment $k \geq 2$, controlling expected GPU memory consumption. The segment structure ensures shorter prompts complete without waiting for longer ones to finish.

EC.3.3. Throughput Gap Bound

Using B as the total batch count (upper bound for each segment's batch index), the throughput gap is:

$$\frac{1}{B}\sum_{k=1}^{m}\mathbb{E}\left[W^B_{(k)}\right] \leq \frac{2n_1}{B} + \frac{\Delta T_{[1,\ldots,m]}\cdot\sum_{j=1}^{m}\lambda_j}{2B\left(n_1 - \Delta T_{[1,\ldots,m]}\cdot\sum_{j=1}^{m}\lambda_j\right)} + \frac{1}{B}\sum_{k=2}^{m} n_k + \frac{1}{B}\sum_{k=2}^{m}\frac{n_{k-1}\cdot p_k(1-p_k)}{2\left(n_k - n_{k-1}\cdot p_k\right)}.$$

Asymptotic Regime. As $\zeta \to \infty$, the throughput gap is $O((\zeta B)^{-1})$. The expected latency and TTFT are $O(1)$.

EC.3.4. High-Probability Memory Bound

High-Probability Bounds Prevent Eviction. Expected queue length bounds ensure average stability. However, stochastic fluctuations may cause $W^b_{(k)}$ to exceed these limits, risking memory overflow and eviction. We derive high-probability bounds to quantify this overflow risk. By choosing capacity C to ensure overflow probability $< \delta$ across all batches and segments, we prevent eviction cascades.

Martingale Construction. To obtain high-probability bounds on queue length, we need to control tail probabilities of the random walk maximum $\max_{0\leq i\leq s-1}\{S^i_{(k)}\}$. Expected bounds alone cannot prevent rare but catastrophic overflow events. We construct an exponential martingale that enables Doob's inequality, converting expected bounds into probabilistic guarantees.

We need to estimate

$$P\left(\max\{0, S^1_{(k)}, \cdots, S^{s-1}_{(k)}\} \geq c\right) = P\left(\max_{0\leq i\leq s-1}\{S^i_{(k)}\} \geq c\right), \forall c > 0, \text{ where } S^0_{(k)} = 0.$$

Define the exponential martingale $M^i_{(k)} = e^{\theta S^i_{(k)}}$. For this to be a martingale, the moment generating function must equal 1, ensuring zero expected drift in log space:

$$\phi_{(k)}(\theta) = \mathbb{E}\left[e^{\theta X^i_{(k)}}\right] = e^{-\theta n_k}(1 - p_k + p_k e^{\theta})^{n_{k-1}} = 1.$$

The following lemma establishes existence and properties of the solution $\theta_k > 0$ that satisfies this martingale condition (proof in Appendix EC.5.5).

LEMMA 5. *Suppose $n_{k-1} > n_k > n_{k-1}p_k$ and $p_k \in (0,1)$. Then there exists a unique solution $\theta_k > 0$ to the equation*

$$e^{-\theta_k n_k}(1 - p_k + p_k e^{\theta_k})^{n_{k-1}} = 1, \tag{18}$$

where θ_k is strictly increasing with respect to $D = n_k - n_{k-1}p_k$ and satisfies the lower bound

$$\theta_k \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}.$$

When $\theta_k > 0$ satisfies $\phi(\theta_k) = 1$, the process $M^i_{(k)}$ is a martingale starting at $M^0_{(k)} = e^{\theta_k \cdot 0} = 1$. This zero-drift property in log space enables Doob's inequality to bound tail probabilities.

By Doob's martingale inequality (Durrett 2019), we have that

$$P(\max_{1\leq i\leq b-1}\{e^{\theta_k S^i_{(k)}}\} \geq a) \leq \frac{\mathbb{E}\left[M^i\right]}{a} = \frac{\mathbb{E}\left[M^0\right]}{a} = \frac{1}{a}.$$

Exponentiating both sides of the inequality event, we obtain:

$$P(\max_{1\leq i\leq b-1}\{e^{\theta S^i_{(k)}}\} \geq e^{\theta c}) \leq \frac{1}{e^{\theta c}} \Rightarrow P(\max_{1\leq i\leq b-1}\{S^i_{(k)}\} \geq c) \leq e^{-\theta c}.$$

Using the dominance $\tilde{W}^b_{(k)} = n_k + \max_{0\leq i\leq b-1}\{S^i_{(k)}\} \geq W^b_{(k)}$, we have

$$P\left(W^b_{(k)} \geq c\right) < P\left(\tilde{W}^b_{(k)} \geq c\right) = P(\max_{1\leq i\leq b-1}\{S^i_{(k)}\} \geq c - n_k) \leq e^{-\theta_k(c-n_k)}$$

For all $2 \leq k \leq m$ and $1 \leq b \leq B$:

$$P\left(W^b_{(k)} \geq c\right) < e^{-\theta_k(c-n_k)},$$

where $\theta_k > 0$ is the solution to (18).

Union bound for safety guarantee. To ensure memory consumption does not exceed the bound at any batch $b \in \{1,\ldots,B\}$ with probability at least $1-\delta$, we apply a union bound across all segments and batches. For each segment $k \in \{2,\ldots,m\}$ and batch index $b \in \{1,\ldots,B\}$, we have:

$$P\left(W^b_{(k)} \geq c\right) < e^{-\theta_k(c-n_k)},$$

There are $m-1$ segments (from $k=2$ to m) and B batches (from $b=1$ to B). We allocate the total failure probability δ across all segments and batches. The total number of events is $(m-1)\cdot B$. Set the overflow probability for each segment and batch to:

$$P\left(W^b_{(k)} \geq c_k\right) \leq \frac{\delta}{(m-1)\cdot B} \Rightarrow e^{-\theta_k(c_k-n_k)} \leq \frac{\delta}{(m-1)\cdot B} \Rightarrow c_k \geq n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}$$

If we choose

$$c_k = n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k},$$

then we have that

$$P\left(W^b_{(k)} \geq c_k\right) \leq \frac{\delta}{(m-1)\cdot B}.$$

Next, we compute the joint probability of overflow across all segments and batches using a union bound:

$$P\left(\bigcup_{k=2}^{m}\bigcup_{b=1}^{B}\{W_{(k)}^{b} \geq c_k\}\right) \leq \sum_{k=2}^{m}\sum_{b=1}^{B} P\left(W_{(k)}^{b} \geq c_k\right) \leq (m-1)\cdot B \cdot \frac{\delta}{(m-1)\cdot B} = \delta$$

Thus:

$$P\left(W_{(k)}^{b} \leq c_k \text{ for all } \begin{matrix} k \in \{2,\ldots,m\}, \\ b \in \{1,\ldots,B\} \end{matrix}\right) \geq 1-\delta.$$

So the memory consumption is bounded:

$$\text{Memory Consumption}^{b} = M^{\pi} + \sum_{k=2}^{m} W_{(k)}^{b}\cdot(l+l'_{k-1}) \leq M^{\pi} + \sum_{k=2}^{m} c_k \cdot (l+l'_{k-1})$$

where M^{π} is the peak batch memory usage as defined in Equation (12):

$$M^{\pi} = \sum_{j=1}^{m} n_j \left(l + \frac{L'_j}{2}\right)\Delta l'_j, \quad \text{with } L'_j = \sum_{i=1}^{j} l'_i,\ \Delta l'_j = l'_j - l'_{j-1},\ l'_0 = 0.$$

Note that this differs from the M^{π} formula for the WAIT algorithm (Theorem 1), because Nested WAIT groups prompts into segments based on their unknown output lengths, leading to segment-wise memory accumulation

Substitute $c_k = n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}$, we have that

$$\text{Memory Consumption}^{b} \leq M^{\pi} + \sum_{k=2}^{m}\left(n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}\right)\cdot(l+l'_{k-1}).$$

Define the memory bound $\mathbf{M}^{B}$:

$$\mathbf{M}^{B} = M^{\pi} + \sum_{k=2}^{m} n_k\cdot(l+l'_{k-1}) + \sum_{k=2}^{m}\frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}\cdot(l+l'_{k-1}).$$

By the lower bound in Lemma 5, we can obtain the upper bound of the third term:

$$\sum_{k=2}^{m}\frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}\cdot(l+l'_{k-1}) \leq \sum_{k=2}^{m}\frac{n_{k-1}}{8(n_k - n_{k-1}p_k)}\ln\left(\frac{(m-1)\cdot B}{\delta}\right)\cdot(l+l'_{k-1})$$

And this $\mathbf{M}^{B}$ ensures that with probability at least $1-\delta$, the memory consumption does not exceed $\mathbf{M}^{B}$ at any batch index $b \in \{1,\ldots,B\}$.

Appendix EC.4: Proof of Theorem 3

Proof Outline. This proof extends Nested WAIT to time-varying arrival rates $\lambda_j(t)$. Time-varying rates make exact load balance impossible, since the optimal threshold depends on the instantaneous arrival pattern. The *worst-case domination strategy* maintains load balance even at peak rates: by designing thresholds to handle worst-case arrivals, we ensure system stability across all scenarios. This conservative approach prevents eviction throughout the time horizon. The structure proceeds as follows:

1. **First segment analysis**: Poisson arrivals with time-varying rates are dominated by a time-invariant process using worst-case aggregate rate $\mathbf{\Lambda}^{\pi} = \sup_b\{\sum_{j=1}^{m}\lambda_j[t(b), t(b)+\Delta T_{[1,\ldots,m]}]\}$. We couple with a Lindley process and apply Kingman's inequality.

2. **Subsequent segments analysis**: Binomial arrivals with time-varying thinning probabilities $p_k(b)$ are dominated by worst-case $p_k^* = \sup_b\{p_k(b)\}$. This ensures safety across all batch compositions. We apply Lindley coupling to each segment.

3. **Throughput and high-probability bounds**: Asymptotic optimality follows from the time-invariant dominated processes. High-probability memory bounds follow the same martingale construction, using worst-case parameters to prevent eviction.

We use b to denote the *batch index* and $k \in \{1,\dots,m\}$ to denote the *segment index*. The total number of batches is denoted by B.

EC.4.1. First Segment Analysis

For the first segment ($k = 1$), the state transition rule is

$$W_{(1)}^{b+1} = W_{(1)}^b + Y_{(1)}^b - n_1 \cdot \mathbf{1}\{W_{(1)}^b + Y_{(1)}^b \geq n_1\}, \quad Y_{(1)}^b \sim \text{Poisson}\Big(\sum_{j=1}^m \lambda_j[t(b), t(b)+\Delta T_b]\Big),$$

where $t(b)$ denotes the start time of batch b and ΔT_b denotes the processing time of batch b.

Worst-case domination. Time-varying arrival rates prevent exact load balance since optimal thresholds depend on instantaneous arrival patterns. To maintain stability across all scenarios, we use worst-case domination: design thresholds to handle peak arrival rates, ensuring eviction prevention even under worst-case conditions. Since $\Delta T_b \leq \Delta T_{[1,\dots,m]}$ (the time for a full batch containing all m types), we construct a dominating process $\{\bar{W}_{(1)}^b\}$:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^b + \bar{Y}_{(1)}^b \geq n_1\}, \quad \bar{Y}_{(1)}^b \sim \text{Poisson}\Big(\sum_{j=1}^m \lambda_j[t(b), t(b)+\Delta T_{[1,\dots,m]}]\Big).$$

By (14), the aggregate arrival rate during any batch is bounded by the worst-case aggregate rate:

$$\sum_{j=1}^m \lambda_j[t(b), t(b)+\Delta T_{[1,\dots,m]}] \leq \sup_{\forall b}\left\{\sum_{j=1}^m \lambda_j[t(b), t(b)+\Delta T_{[1,\dots,m]}]\right\} < n_1.$$

Define $\mathbf{\Lambda}^\pi = \sup_{\forall b}\left\{\sum_{j=1}^m \lambda_j[t(b), t(b)+\Delta T_{[1,\dots,m]}]\right\} < n_1$ as the worst-case aggregate arrival rate. This enables a time-invariant dominating process that provides tractable bounds:

$$\tilde{W}_{(1)}^{b+1} = \tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b \geq n_1\}, \quad \tilde{Y}_{(1)}^b \sim \text{Poisson}(\mathbf{\Lambda}^\pi).$$

This time-invariant process enables classical queueing analysis. Define $X_{(1)}^b = \tilde{Y}_{(1)}^b - n_1$. The process $\{\tilde{W}_{(1)}^b\}$ is dominated by a Lindley process:

$$\overline{W}_{(1)}^0 = 2n_1, \quad \overline{W}_{(1)}^{b+1} = \max\{2n_1, \overline{W}_{(1)}^b + X_{(1)}^b\}, \quad \forall b \in \mathbb{N}.$$

Thus we have the dominance chain:

$$W_{(1)}^b \leq \bar{W}_{(1)}^b \leq \tilde{W}_{(1)}^b \leq \overline{W}_{(1)}^b, \quad \forall b \in \mathbb{N}.$$

Since $n_1 > \mathbf{\Lambda}^\pi$, the process $\{\overline{W}_{(1)}^b\}$ has negative drift. By Kingman's inequality (Kingman 1962), with

$$\text{Var}(X_{(1)}^b) = \mathbf{\Lambda}^\pi, \quad \left|\mathbb{E}\left[X_{(1)}^b\right]\right| = n_1 - \mathbf{\Lambda}^\pi,$$

we obtain

$$\mathbb{E}\left[W_{(1)}^b\right] \leq 2n_1 + \frac{\mathbf{\Lambda}^\pi}{2\,(n_1 - \mathbf{\Lambda}^\pi)}.$$

EC.4.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), arrivals come from prompts that completed segment $k-1$. The batch index b for segment k counts only batches containing segment $k-1$. These indices differ across segments but are bounded by the total batch count B.

Time-varying thinning probabilities. Consider the b-th arrival to segment k, which comes from the b-th batch containing segment $k-1$. The thinning probability $p_k(b)$ depends on the prompt composition in this batch and varies over time. Although exact arrival times are unknown, arrivals occur within some interval $[t, t+\Delta t]$ where $t+\Delta t \leq t(b, k-1)$, the starting time of the b-th batch containing segment $k-1$. To handle this time variation, we use the worst-case thinning probability. By (14):

$$p_k(b) = \frac{\sum_{j=k+1}^{m} \lambda_j[t, t+\Delta t]}{\sum_{j=k}^{m} \lambda_j[t, t+\Delta t]} \leq p_k^* < \frac{n_k}{n_{k-1}}, \quad \forall b \in \mathbb{N}.$$

The arrival process is

$$Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b)), \quad \forall b \geq 0,$$

and the state transition rule is

$$W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k \cdot \mathbf{1}\{W_{(k)}^b + Y_{(k)}^b \geq n_k\},$$

where b indexes batches containing segment $k-1$, since new arrivals to segment k occur only when segment $k-1$ is processed.

Define $X_{(k)}^b = Y_{(k)}^b - n_k$. Although the distribution $Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b))$ is time-varying, the coupling construction follows Lemma 4:

LEMMA 6 (**Coupling for Time-Varying Case**). *Define a coupled process* $\{\tilde{W}_{(k)}^b\}$ *by:*

$$\begin{aligned}
\tilde{W}_{(k)}^0 &= n_k, \\
\tilde{W}_{(k)}^{b+1} &= \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\}, \\
X_{(k)}^b &= Y_{(k)}^b - n_k.
\end{aligned}$$

Then $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ *for all* $b \in \mathbb{N}$.

To apply Kingman's inequality (Kingman 1962), we construct a time-invariant dominating process:

$$\begin{aligned}
\bar{W}_{(k)}^0 &= n_k, \\
\bar{W}_{(k)}^{b+1} &= \max\{n_k, \bar{W}_{(k)}^b + \bar{X}_{(k)}^b\}, \\
\bar{X}_{(k)}^b &= \bar{Y}_{(k)}^b - n_k, \\
\bar{Y}_{(k)}^b &\sim \text{Binomial}(n_{k-1}, p_k^*), \quad \forall b \in \mathbb{N}.
\end{aligned}$$

Then we have the dominance chain:

$$\bar{W}_{(k)}^b \geq \tilde{W}_{(k)}^b \geq W_{(k)}^b, \quad \forall b \in \mathbb{N}.$$

The process $\bar{W}^b_{(k)}$ has the Lindley representation:

$$\bar{W}^0_{(k)} = n_k,$$
$$\bar{W}^{b+1}_{(k)} = n_k + \max_{0\le i\le b}\{\bar{S}^i_{(k)}\},$$
$$\text{where } \bar{S}^0_{(k)} = 0,\ \bar{S}^i_{(k)} = \sum_{r=1}^{i} \bar{X}^r_{(k)},\ 1 \le i \le b.$$

Since

$$\mathbb{E}\left[\bar{X}^b_{(k)}\right] = n_{k-1}\cdot p^*_k - n_k < 0, \quad \forall b \in \mathbb{N},$$

the coupled process $\{\bar{W}^b_{(k)}\}$ has negative drift. With

$$\mathrm{Var}(\bar{X}^b_{(k)}) = n_{k-1}p^*_k(1-p^*_k), \quad \left|\mathbb{E}\left[\bar{X}^b_{(k)}\right]\right| = n_k - p^*_k n_{k-1},$$

Kingman's inequality (Kingman 1962) yields:

$$\mathbb{E}\left[\max_{0\le i\le b}\{\bar{S}^i_{(k)}\}\right] \le \frac{n_{k-1}p^*_k(1-p^*_k)}{2(n_k - n_{k-1}p^*_k)}.$$

Thus, the expected queue length is bounded:

$$\mathbb{E}\left[W^b_{(k)}\right] \le \mathbb{E}\left[\tilde{W}^b_{(k)}\right] \le \mathbb{E}\left[\bar{W}^b_{(k)}\right] \le n_k + \frac{n_{k-1}p^*_k(1-p^*_k)}{2(n_k - n_{k-1}p^*_k)}.$$

EC.4.3. Throughput and High-Probability Bounds

The discussion of asymptotic optimal throughput under bounded latency and TTFT follows Appendix EC.3. For high-probability bounds, the analysis follows the same martingale construction applied to the time-invariant coupled process $\{\bar{W}^b_{(k)}\}$ defined above. By the same argument as in Appendix EC.3, with $\bar{X}^r_{(k)} = \bar{Y}^r_{(k)} - n_k$, $\bar{Y}^r_{(k)} \sim \text{Binomial}(n_{k-1}, p^*_k)$, and $p^*_k < n_k/n_{k-1}$, the memory bound $\mathbf{M}^B$ is:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^{m} n_k\cdot(l + l'_{k-1}) + \sum_{k=2}^{m}\theta_k^{-1}\ln\left(\frac{(m-1)\cdot B}{\delta}\right)\cdot(l + l'_{k-1}),$$

where θ_k $(2 \le k \le m)$ is the unique positive solution to:

$$e^{-\theta_k n_k}(1 - p^*_k + p^*_k e^{\theta_k})^{n_{k-1}} = 1. \tag{19}$$

Since $p^*_k = \sup_b\{p_k(b)\}$ represents the worst-case thinning probability, by the strictly increasing property of θ_k in Lemma 5, this yields the smallest θ_k and hence the largest memory bound term. The lower bound for θ_k follows from the same analysis in Appendix EC.3.

Appendix EC.5: Proof of Lemmas

Throughout this section, we use b for the batch index and B for the total number of batches.

EC.5.1. Proof of Lemma 1

To prove this lemma, we take expectations on both sides of the state transition equation:

$$\mathbb{E}\left[W^{b+1}\right] = \mathbb{E}\left[W^b\right] + \lambda - \lambda\mathbb{P}(W^b + X^b \ge \lambda).$$

Summing from $b = 0$ to $B-1$, and noting that $W^0 = 0$, we obtain:

$$\mathbb{E}\left[W^B\right] = \lambda B - \lambda\mathbb{E}\left[B - B_{\text{stuck}}\right] = \lambda\cdot\mathbb{E}\left[B_{\text{stuck}}\right].$$

EC.5.2. Proof of Lemma 2

We prove by induction on the batch index b. The coupled process $\{\tilde{W}^b\}$ starts with a safety buffer of 2λ (compared to the real process starting at 0), ensuring it remains above $W^b + \lambda$ throughout. For the inductive step, we consider two cases based on whether the system can process a batch or experiences a stuck iteration.

Base case. For $b = 0$: $\tilde{W}^0 = 2\lambda \geq \lambda = W^0 + \lambda$.

Inductive step. Assume $\tilde{W}^b \geq W^b + \lambda$ for some $b \geq 0$. We show $\tilde{W}^{b+1} \geq W^{b+1} + \lambda$ by considering two cases:

Case 1 (Batch processed): If $W^b + X^b \geq \lambda$, the queue has accumulated enough prompts to process a batch, so $W^{b+1} = W^b + X^b - \lambda$. Then

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq \tilde{W}^b + X^b - \lambda \geq (W^b + \lambda) + X^b - \lambda = W^{b+1} + \lambda.$$

Case 2 (Stuck iteration): If $W^b + X^b < \lambda$, insufficient prompts are available and the system cannot process a batch (stuck iteration). The queue simply accumulates arrivals: $W^{b+1} = W^b + X^b$. The safety buffer ensures dominance:

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq 2\lambda = \lambda + \lambda > W^{b+1} + \lambda.$$

By induction, $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

EC.5.3. Proof of Lemma 3

Proof idea. The coupled process dominates the real process by maintaining a consistent full-batch processing schedule, while the real process may skip type j in some batches when $W^b_{(j)} < n_j$. We analyze periods when type j actively joins batches and show the coupled process makes at most one additional batch iteration, preserving dominance.

Proof. Since $\tilde{W}^b_{(j)} \geq 2n_j$, when $W^b_{(j)} < n_j$, we trivially have $\tilde{W}^b_{(j)} \geq W^b_{(j)}$.

Initially, $W^0_{(j)} = 0$. As arrivals accumulate, $W^b_{(j)}$ reaches n_j and type j joins the batch. After processing, $W^b_{(j)}$ may drop below n_j, and the cycle repeats.

Consider a period $[b_0, b_1]$ when $W^b_{(j)} \geq n_j$, meaning type j is actively joining batches in the real process. Let $\tau = \max\{b \leq b_0 : W^b_{(j)} = n_j\}$ be the last time before b_0 when the queue length equals exactly n_j. We compare the number of batch iterations between the real and coupled processes over the interval $[\tau, b]$.

For any $b \in [b_0, b_1]$, the elapsed time is $t(b) - t(\tau)$, where $t(b)$ denotes the time at batch b. Since each batch takes at most $\Delta T_{[1,\cdots,m]}$ time (the full-batch processing time), we can bound the number of batch iterations $I_{\tau\to b}$ (real process) and $\tilde{I}_{\tau\to b}$ (coupled process):

$$I_{\tau\to b} \geq \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1,\cdots,m]}} \rfloor, \quad \tilde{I}_{\tau\to b} \leq 1 + \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1,\cdots,m]}} \rfloor \Rightarrow \tilde{I}_{\tau\to b} \leq 1 + I_{\tau\to b}.$$

Thus, the coupled process makes at most one more iteration than the real process after τ.

At time τ, the real process may be in a batch without type j (since not all batches contain all types), while the coupled process just completed a full batch containing all types. This yields the initial dominance $\tilde{W}^\tau_{(j)} \geq n_j = W^\tau_{(j)}$.

For batches in the interval $(\tau, b]$, the arrival dynamics are identical for both processes. Since type j joins every batch in the real process during this period (as $W^b_{(j)} \geq n_j$), but the coupled process completes at most one additional batch, the dominance is maintained. Thus $\tilde{W}^b_{(j)} \geq W^b_{(j)}$ for all $b \geq 0$.

EC.5.4. Proof of Lemma 4

The proof follows the same induction structure as Lemma 2, adapted to the Nested WAIT setting where arrivals follow binomial thinning from segment $k-1$ completions.

Base case. For $b=0$: $\tilde{W}^0_{(k)} = n_k \geq 0 = W^0_{(k)}$.

Inductive step. Assume $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ for some $b \geq 0$. We show $\tilde{W}^{b+1}_{(k)} \geq W^{b+1}_{(k)}$ by case analysis:

Case 1 (Segment batch processed): If $W^b_{(k)} + Y^b_{(k)} \geq n_k$, then $W^{b+1}_{(k)} = W^b_{(k)} + Y^b_{(k)} - n_k$, so

$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\} \geq \tilde{W}^b_{(k)} + X^b_{(k)} \geq W^b_{(k)} + Y^b_{(k)} - n_k = W^{b+1}_{(k)}.$$

Case 2 (Segment stuck): If $W^b_{(k)} + Y^b_{(k)} < n_k$, then $W^{b+1}_{(k)} = W^b_{(k)} + Y^b_{(k)}$, so

$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\} \geq n_k > W^{b+1}_{(k)}.$$

By induction, $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ for all $b \in \mathbb{N}$.

EC.5.5. Proof of Lemma 5

This lemma establishes existence and properties of the parameter $\theta_k > 0$ that makes the exponential process $e^{\theta_k S^i_{(k)}}$ a martingale. This enables Doob's inequality to convert expected queue length bounds into high-probability bounds, preventing eviction cascades.

Existence and uniqueness. For simplicity, we denote $\theta = \theta_k$. Define the function

$$f(\theta) = e^{-\theta n_k}(1 - p_k + p_k e^{\theta})^{n_{k-1}}.$$

We seek $\theta > 0$ such that $f(\theta) = 1$. First, evaluate $f(\theta)$ at $\theta = 0$:

$$f(0) = e^0(1 - p_k + p_k \cdot 1)^{n_{k-1}} = 1.$$

Thus, $\theta = 0$ is a trivial solution, but we need $\theta > 0$ to obtain useful high-probability bounds. To analyze $f(\theta)$ for $\theta > 0$, we consider the logarithm

$$g(\theta) = \ln f(\theta) = -\theta n_k + n_{k-1} \ln(1 - p_k + p_k e^{\theta}),$$
$$g'(\theta) = -n_k + n_{k-1} \cdot \frac{p_k e^{\theta}}{1 - p_k + p_k e^{\theta}}.$$

At $\theta = 0$,

$$g(0) = 0,\ g'(0) = -n_k + n_{k-1} p_k.$$

Since $\mathbb{E}[X^b_{(k)}] = n_{k-1} p_k - n_k < 0$, we have $g'(0) < 0$, so $f(\theta)$ is decreasing near $\theta = 0$. Consider the second derivative:

$$g''(\theta) = n_{k-1} \cdot \frac{p_k e^{\theta}(1 - p_k)}{(1 - p_k + p_k e^{\theta})^2} > 0 \quad \text{for} \quad \theta > 0,$$

since $p_k \in (0,1)$ and $n_{k-1} > 0$. Thus, $g(\theta)$ is strictly convex. Convexity ensures that $g(\theta)$ can cross zero at most once for $\theta > 0$, guaranteeing uniqueness. Now, consider the limit as $\theta \to \infty$:

$$f(\theta) \approx p_k^{n_{k-1}} e^{\theta(n_{k-1} - n_k)}.$$

By our settings, $n_{k-1} > n_k$, which ensures $f(\theta) \to \infty$ as $\theta \to \infty$. Thus, $f(\theta)$ is continuous, convex, starts at $f(0) = 1$, decreases below 1 for small $\theta > 0$ (since $g'(0) = n_{k-1} p_k - n_k < 0$ by negative drift), and increases to infinity. By the intermediate value theorem and convexity, there exists a unique $\theta > 0$ such that $f(\theta) = 1$.

Monotonicity in drift. Next, we prove that the solution $\theta > 0$ is strictly increasing with respect to the drift $D = n_k - n_{k-1}p_k$. This shows that tighter capacity (larger D, more negative drift) leads to larger θ and thus tighter high-probability bounds. Recall that $g(0) = 0$, $g'(0) = -D < 0$, and $g''(\theta) > 0$ for $\theta > 0$, so $g(\theta)$ is strictly convex. The unique positive solution $\theta > 0$ occurs where $g(\theta)$ crosses zero after initially decreasing from $g(0) = 0$.

Consider two values D_1 and D_2 such that $D_1 < D_2$, with corresponding functions $g_1(\theta) = -\theta(n_{k-1}p_k + D_1) + n_{k-1}\ln(1 - p_k + p_k e^{\theta})$ and $g_2(\theta) = -\theta(n_{k-1}p_k + D_2) + n_{k-1}\ln(1 - p_k + p_k e^{\theta})$, and solutions θ_1 and θ_2, respectively. At $\theta = 0$, we have $g_1'(0) = -D_1 > g_2'(0) = -D_2$, meaning $g_2(\theta)$ decreases more steeply from zero than $g_1(\theta)$.

For a fixed $\theta > 0$, the partial derivative $\frac{\partial g}{\partial D} = -\theta < 0$, so increasing D decreases $g(\theta)$ pointwise. Since $g(\theta)$ is convex and approaches infinity as $\theta \to \infty$, the zero crossing of $g(\theta; D)$ shifts to a larger θ when D increases. Hence, if $D_2 > D_1$, then $\theta_2 > \theta_1$. Operationally, this means larger safety margins yield tighter high-probability bounds.

Drift-to-θ scaling. Finally, we characterize how the solution $\theta > 0$ scales with drift D, n_{k-1}, and p_k. When the drift $D = n_k - n_{k-1}p_k$ is small (system near critical load), the solution θ is also small. To derive an explicit approximation, we expand $g(\theta)$ around $\theta = 0$ using Taylor's theorem:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(0)\theta^2 + O(\theta^3) = 0 - D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 + O(\theta^3).$$

Setting $g(\theta) = 0$ and neglecting higher-order terms, we obtain the quadratic approximation

$$-D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 \approx 0 \implies \theta \approx \frac{2D}{n_{k-1}p_k(1 - p_k)}.$$

This shows that for small D, the solution scales linearly: $\theta = O(D)$. Specifically, $\theta \approx \frac{2(n_k - n_{k-1}p_k)}{n_{k-1}p_k(1-p_k)}$. This linear scaling implies that the high-probability memory bound grows logarithmically in B/δ, with the coefficient inversely proportional to the drift D.

General lower bound. To obtain a uniform lower bound valid for all $D > 0$ (not just small D), we apply Taylor's theorem with remainder:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(c)\theta^2 = -D\theta + \frac{1}{2}g''(c)\theta^2 = 0 \implies \theta = \frac{2D}{g''(c)}. \quad \text{for some } c \in (0, \theta)$$

To find an upper bound for $g''(c)$, we write $g''(x) = n_{k-1} \cdot \frac{p_k e^x (1-p_k)}{(1-p_k+p_k e^x)^2}$ and define $h(y) = \frac{p_k y(1-p_k)}{(1-p_k+p_k y)^2}$ where $y = e^x \geq 1$. By differentiation, $h(y)$ achieves a maximum of $\frac{1}{4}$ over $y \geq 1$ and $0 < p_k < 1$ (attained when the numerator and denominator are balanced). Thus we have that $g''(x) \leq n_{k-1} \cdot \frac{1}{4} = \frac{n_{k-1}}{4} \implies \theta = \frac{2D}{g''(c)} \geq \frac{2D}{n_{k-1}/4} = \frac{8D}{n_{k-1}}$. So a general lower bound is:

$$\theta \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}. \tag{20}$$